# DOCUMENTO DE DISEÑO

# CLUB DEPORTIVO SÉNECA

PROYECTO 1, ENTREGA 2

Grupo 4
Manuela Leon 202520087
Samuel Fonseca 202521098
Camilo Castro 202412178

Diseño y Programación Orientada a Objetos (ISIS-1226)
Universidad de los Andes

# 1. CONTEXTO Y ALCANCE

### EL PROBLEMA

El Club Deportivo Séneca necesita un sistema para su parte deportiva y su parte comercial. En lo deportivo maneja equipos de fútbol, baloncesto y voleibol por categoría de edad, partidos internos con estadísticas y sanciones, torneos externos (Torneo de los Cerros y ASCUN), las academias de tenis, pádel y tenis de mesa con reservas, niveles y ranking, y las sesiones de entrenamiento con su asistencia. En lo comercial maneja la tienda y la cafetería, cada una con su impuesto, los descuentos de empleado, los puntos de fidelidad, los alérgenos, el inventario por ubicación, los turnos del personal y los reportes.

Lo usan tres tipos de usuario: el **socio** (usuario final), el **empleado**, que es entrenador o fisioterapeuta (usuario operativo), y el **administrador**.

### REQUERIMIENTOS FUNCIONALES

Son los del análisis de la Entrega 1. Para cada uno, la tabla dice **dónde sale en el código**: el método que lo resuelve y el menú donde se prueba. Con esta tabla se puede abrir Eclipse e ir directo a cada requerimiento.

| RF | Qué pide | Dónde se resuelve | Dónde se prueba |
|---|---|---|---|
| **RF-S-01** | Autenticarse | `Club.autenticar` | AplicacionClub (login) |
| **RF-S-02** | Consultar los equipos y disciplinas en los que está inscrito | `PerfilJugador.consultarEquipos` | MenuSocio |
| **RF-S-03** | Inscribirse en un equipo de su categoría | `Equipo.inscribirJugador` | MenuSocio |
| **RF-S-04** | Consultar el calendario de partidos | `Club.consultarCalendario` | MenuSocio |
| **RF-S-05** | Consultar sus estadísticas | `Club.consultarEstadisticas` | MenuSocio |
| **RF-S-06** | Consultar sus sanciones vigentes | `PerfilJugador.consultarSancionesVigentes` | MenuSocio |
| **RF-S-07** | Inscribirse en una disciplina individual | `DisciplinaIndividual.inscribir` | MenuSocio |
| **RF-S-08** | Reservar una instalación | `DisciplinaIndividual.reservar` | MenuSocio |
| **RF-S-09** | Programar un partido de práctica | `DisciplinaIndividual.programarPartidoPractica` | MenuSocio |
| **RF-S-10** | Consultar su ranking y su evolución | `InscripcionDisciplina.getPuntajeRanking`<br>`PartidoPractica.cerrarYActualizarRanking`<br>`GeneradorReportes.evolucionDeRanking` | MenuSocio |
| **RF-S-11** | Consultar sus convocatorias y asistencias | `GeneradorReportes.convocatoriasDeJugador` | MenuSocio |
| **RF-S-12** | Comprar en la tienda | `Club.registrarVenta` con una `VentaTienda` | MenuSocio |
| **RF-S-13** | Comprar en la cafetería | `Club.registrarVenta` con una `VentaCafeteria` | MenuSocio |
| **RF-S-14** | Consultar y redimir puntos | `Venta.cerrar` y `Usuario.redimirPuntos` | MenuSocio |

| RF | Qué pide | Dónde se resuelve | Dónde se prueba |
|---|---|---|---|
| **RF-E-01** | Autenticarse | `Club.autenticar` | AplicacionClub (login) |
| **RF-E-02** | Crear una sesión y convocar | `SesionEntrenamiento.convocar` | MenuEmpleado |
| **RF-E-03** | Registrar la asistencia | `SesionEntrenamiento.registrarAsistencia` y `cerrar` | MenuEmpleado |
| **RF-E-04** | Registrar la evaluación de desempeño | `SesionEntrenamiento.registrarAsistencia` (la nota) | MenuEmpleado |
| **RF-E-05** | Registrar el partido y sus estadísticas | `Club.programarPartidoInterno` `Partido.registrarEstadistica` y `Partido.cerrar` | MenuEmpleado |
| **RF-E-06** | Registrar tarjetas y suspensiones | `PartidoInterno.despuesDeCerrar` `InscripcionEquipo.registrarTarjetas` | MenuEmpleado |
| **RF-E-07** | Verificar el mínimo de habilitados | `Equipo.puedeDisputarPartido` | MenuEmpleado |
| **RF-E-08** | Proponer la nómina de jugadores para inscribir un equipo en una competencia | `Competencia.inscribirEquipo` | MenuAdministrador |
| **RF-E-09** | Consultar asistencia y desempeño | `GeneradorReportes.asistenciaDeJugador` | MenuEmpleado |
| **RF-E-10** | Solicitar un cambio de turno | `Club.solicitarCambioTurno` | MenuEmpleado |
| **RF-E-11** | Sugerir artículos o snacks | `Club.registrarSugerencia` | MenuEmpleado |
| **RF-E-12** | Comprar con código de descuento | `Venta.aplicarCodigoDescuento` | MenuEmpleado |
| **RF-E-13** | Habilitación médica | No implementado. Queda fuera de esta entrega: es una limitación. | No |
| **RF-A-01** | Autenticarse | `Club.autenticar` | AplicacionClub (login) |
| **RF-A-02** | Crear y dar de baja equipos y jugadores | `Club.crearEquipo`, `Club.darDeBajaEquipo`, `Club.darDeBajaJugador` | MenuAdministrador |
| **RF-A-03** | Crear competencias e inscribir equipos | `Competencia.inscribirEquipo`, `programarPartido`, `registrarResultado` | MenuAdministrador |
| **RF-A-04** | Consultar el inventario | `Club.consultarInventario` | MenuAdministrador |
| **RF-A-05** | Mover y reabastecer artículos | `Club.moverProducto` y `Club.reabastecer` | MenuAdministrador |
| **RF-A-06** | Aprobar o rechazar sugerencias | `SugerenciaProducto.aprobar` y `rechazar` | MenuAdministrador |
| **RF-A-07** | Consultar el historial de partidos | `GeneradorReportes.resultadosDeEquipo` | MenuAdministrador |
| **RF-A-08** | Consultar la cobertura de turnos del personal | `Club.hayCobertura` | MenuAdministrador |
| **RF-A-09** | Aprobar o rechazar cambios de turno | `Club.resolverCambioTurno` | MenuAdministrador |

| RF | Qué pide | Dónde se resuelve | Dónde se prueba |
|---|---|---|---|
| **RF-A-10** | Reporte de ventas por rubro | `GeneradorReportes.ventasPorRubro` | MenuAdministrador |
| **RF-A-11** | Consultar el reporte de asistencia de un jugador | `GeneradorReportes.asistenciaDeJugador` | MenuAdministrador |
| **RF-A-12** | Estado de las instalaciones | `DisciplinaIndividual.consultarInstalacionesOcupadas` | MenuAdministrador |

Las reglas de dominio (RN-01 a RN-38) también vienen del análisis. Encima de cada método, el comentario dice qué requerimiento resuelve y qué reglas revisa. Cuando algo se rechaza, el mensaje lleva el número de la regla, por ejemplo `[RN-12] Sofia Herrera no puede jugar el partido PI-1.`

## REQUERIMIENTOS NO FUNCIONALES Y RESTRICCIONES

Para cada uno: qué pide y cómo lo hacemos en el código.

| Requerimiento | Qué pide | Cómo lo hacemos |
|---|---|---|
| **Lenguaje** | Java, sin librerías externas. | Solo usamos lo que trae Java: `java.util` para las listas y los mapas, `java.time` para fechas y horas y `java.io` para el archivo. |
| **Persistencia** | Guardar el estado en archivos, fuera de la carpeta del código. | `PersistenciaClub` guarda la información mediante registros de texto y la reconstruye al cargarla. La aplicación utiliza el archivo datos/club.txt y contempla datos/club_ejemplo.txt como archivo alternativo. La lectura y escritura se centralizan en PersistenciaClub (sección 8, D5). |
| **Autenticación** | Todo usuario entra con login y clave. | `Club.autenticar` busca el login y `Usuario.validarPassword` compara la clave. Si algo falla, el mensaje es el mismo para no revelar qué logins existen. |
| **Facilidad de cambio** | Cambiar la interfaz o el archivo no debe obligar a tocar las reglas. | El modelo no imprime nada ni sabe de archivos. Lo que cambia según el tipo (partido, venta, instalación) está en subclases. |
| **Mensajes claros** | Si algo no se puede hacer, decir por qué. | Cada rechazo es una excepción con el número de la regla que se rompió (sección 8, D4). |
| **Pruebas** | Mostrar que cada requerimiento funciona. | Recorrer los tres menús (socio, empleado y administrador) con los usuarios de datos/club_ejemplo.txt y comparar cada resultado con lo que pide el requerimiento. La prueba es manual: el repositorio no tiene pruebas automáticas. El recorrido de los tres usuarios está grabado en el video de demostración: https://www.youtube.com/watch?v=j3xLtcEozh4 |

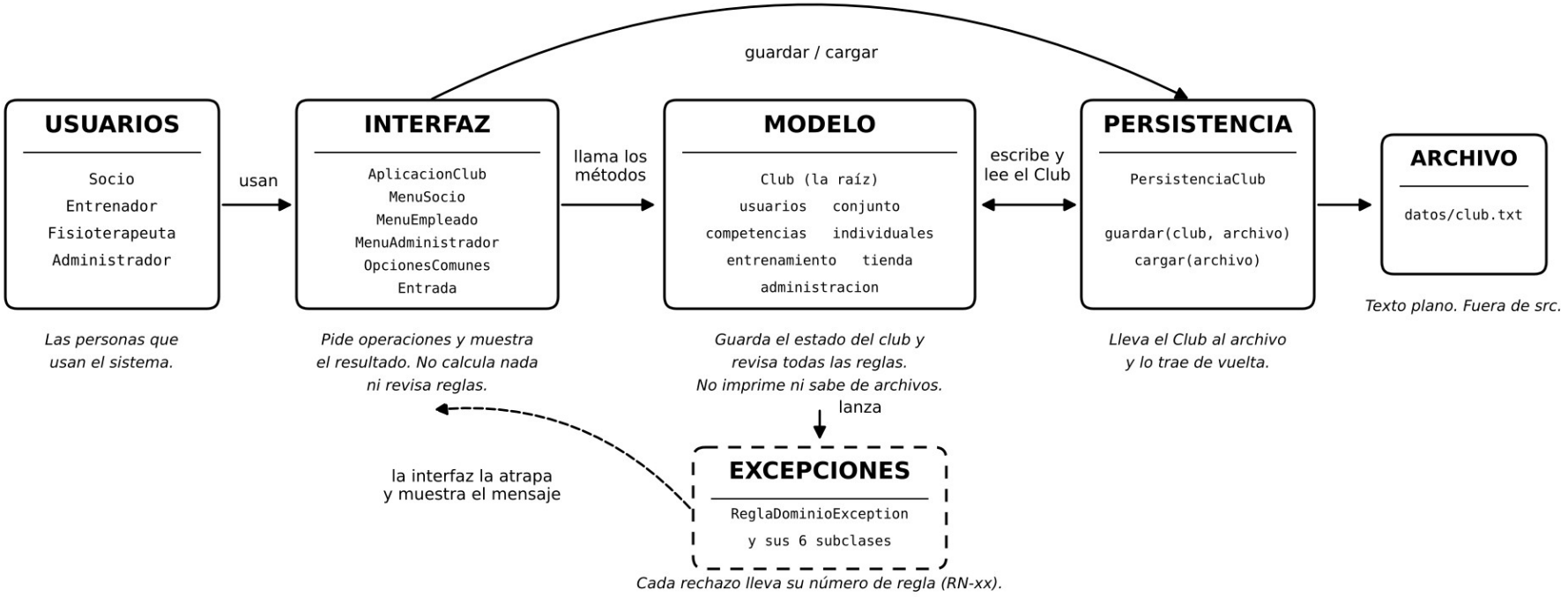
## OBJETIVOS Y NO-OBJETIVOS

Lo que este proyecto sí resuelve y lo que dejamos por fuera a propósito, para no gastar tiempo en algo que no se pidió.

| Objetivos | No-objetivos |
| --- | --- |
| • El modelo completo con las reglas del análisis<br>• Comprobar cada requerimiento a mano, con los menús y los datos de ejemplo, y ver que cada caso que debe rechazarse muestre la regla incumplida<br>• Guardar y cargar todo el club<br>• Que cada rechazo diga qué regla se rompió | • Interfaz gráfica (Proyecto 3)<br>• Pagos reales y facturación electrónica<br>• Modelar a los rivales externos: de ellos solo guardamos el nombre<br>• La tabla completa de los torneos: solo están los equipos del club<br>• Saber qué lleva alguien en la mano: de la bebida caliente solo controlamos la venta<br>• Cifrar las contraseñas |

## 2. VISIÓN GENERAL

El sistema se parte en cuatro componentes y cada uno se explica en una frase. Los usuarios solo tocan la interfaz (los menús de consola). La interfaz le pide las operaciones al modelo. El modelo es el único que conoce las reglas. La persistencia es la única que sabe del archivo. Cuando una regla no se cumple, el modelo lanza una excepción y la interfaz la muestra.




### POR QUÉ LO PARTIMOS ASÍ

- **Si cambia la interfaz, solo cambian los menús del paquete interfaz. Si el proyecto pasa a una interfaz gráfica, el modelo no cambia.** Las reglas no se tocan, porque el modelo no imprime nada.
- **Si cambia el archivo, solo cambia la persistencia.** Si mañana el club se guarda en una base de datos o en otro formato, solo se reescribe PersistenciaClub.
- **El modelo se parte igual que el análisis.** Así cada integrante puede conocer bien una parte, y el código se compara fácil con el modelo de la Entrega 1.

## LOS PAQUETES DEL MODELO

| Paquete | Qué tiene |
|---|---|
| **usuarios** | Quién entra al sistema y quién puede jugar. |
| **conjunto** | Equipos, categorías, inscripciones, partidos internos, estadísticas y sanciones. |
| **competencias** | Torneos externos (liga y por fases), nóminas y tablas de posiciones. |
| **individuales** | Tenis, pádel y tenis de mesa: niveles, instalaciones, reservas, partidos de práctica y ranking. |
| **entrenamiento** | Sesiones y asistencia. |
| **tienda** | Productos, inventario, ventas, descuentos y puntos. |
| **administracion** | El Club, de donde cuelga todo, los turnos y los reportes. |

## 3. ROLES Y CANDIDATOS

Cada clase con una descripción de una línea y su estereotipo: **information holder** (sabe cosas de sí misma), **structurer** (organiza otros objetos), **service provider** (hace un trabajo cuando se lo piden), **interfacer** (traduce entre el sistema y algo de afuera), **coordinator** (reparte el trabajo) y **controller** (decide el flujo).

### USUARIOS

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **Usuario** | Persona que entra al sistema: login, clave, nombre, fecha de nacimiento y puntos | information holder |
| **Socio, Entrenador, Fisioterapeuta, Administrador** | Los tipos de usuario; cada uno dice su rol | information holder |
| **Empleado** | Lo común de entrenador y fisioterapeuta: turnos y código de descuento | information holder |
| **PerfilJugador** | La participación de un usuario como jugador; agrupa sus inscripciones | structurer |

### CONJUNTO

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **DeporteConjunto** | Fútbol, baloncesto o voleibol con su mínimo de jugadores y sus puntos | information holder |
| **Categoria** | Sub-13, Sub-15, Sub-17 o Mayores con su rango de edad | information holder |
| **Equipo** | Un equipo de un deporte y una categoría; decide quién se puede inscribir | structurer |
| **InscripcionEquipo** | Un jugador en un equipo, con su posición, camiseta, amarillas y sanciones | information holder |
| **Sancion** | Una suspensión con su motivo y los partidos que faltan | information holder |
| **Partido** | Lo común de todo partido: fecha, marcador y estadísticas | structurer |
| **PartidoInterno** | Partido entre dos equipos del club | structurer |
| **EstadisticaPartido y sus 3 subclases de conjunto** | Lo que hizo un jugador en un partido, según el deporte | information holder |

### COMPETENCIAS

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **Competencia** | Un torneo externo de un deporte y una categoría | structurer |
| **CompetenciaLiga, CompetenciaPorFases** | Los dos formatos; cada uno registra resultados a su manera | structurer |

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **InscripcionCompetencia** | Un equipo inscrito en un torneo, con su nómina | information holder |
| **NominaCompetencia** | Los jugadores que el torneo deja jugar | structurer |
| **RenglonNomina** | Un jugador dentro de una nómina | information holder |
| **PartidoCompetencia** | Partido oficial contra otra institución | structurer |
| **TablaPosiciones** | La tabla de una liga o de un grupo | structurer |
| **FilaTabla** | La línea de un equipo en una tabla | information holder |

**INDIVIDUALES**

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **DisciplinaIndividual** | Tenis, pádel o tenis de mesa; decide reservas y partidos de práctica | structurer |
| **InscripcionDisciplina** | Un jugador en una disciplina, con su nivel y su ranking | information holder |
| **Instalacion y sus 3 subclases** | Cancha o mesa; cada una dice qué modalidad acepta y si permite bebida caliente | information holder |
| **Reserva** | Una instalación ocupada por un socio en una franja | information holder |
| **PartidoPractica** | Partido entre socios sobre una reserva; ajusta el ranking | structurer |
| **AjusteRanking** | Un cambio de ranking con el antes y el después | information holder |
| **EstadisticaRaqueta, EstadisticaTenisMesa** | Lo que hizo un jugador en un partido de práctica | information holder |

**ENTRENAMIENTO**

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **SesionEntrenamiento** | Una sesión de un equipo o una disciplina, con sus convocados | structurer |
| **RegistroAsistencia** | La asistencia y la nota de un convocado | information holder |

**TIENDA**

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **Producto y sus subclases** | Lo que se vende: artículos deportivos, bebidas y snacks | information holder |
| **Alergeno** | Un alérgeno de un snack | information holder |

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **UbicacionInventario** | Bodega, tienda o cafetería con sus existencias | structurer |
| **Existencia** | Cuántas unidades de un producto hay en una ubicación | information holder |
| **MovimientoInventario** | Un traslado, reabastecimiento o venta de mercancía | information holder |
| **Venta, VentaTienda, VentaCafeteria** | Una compra con sus líneas; calcula su total | structurer |
| **LineaVenta** | Un producto dentro de una venta, con el precio del momento | information holder |
| **CodigoDescuento** | El código de un empleado y sus dos porcentajes | information holder |
| **MovimientoPuntos** | Puntos ganados o usados en una venta | information holder |
| **SugerenciaProducto** | La propuesta de un empleado y quién la resolvió | information holder |
| **PagoMensualidad** | La cuota de un socio | information holder |

### ADMINISTRACION

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **Club** | Tiene las listas de todo y coordina las operaciones que cruzan partes del modelo | coordinator |
| **Turno** | Un día y una franja de trabajo de un empleado | information holder |
| **SolicitudCambioTurno** | El pedido de ceder un turno y cómo se resolvió | information holder |
| **GeneradorReportes** | Calcula los reportes a partir de lo que tiene el club | service provider |

### OTROS

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **Las 7 excepciones** | Un rechazo con su mensaje y su número de regla | information holder |
| **PersistenciaClub** | Lleva el club al archivo y lo trae de vuelta | interfacer |
| **MenuSocio, MenuEmpleado, MenuAdministrador, OpcionesComunes** | Muestran las operaciones de cada tipo de usuario y las envían al modelo. OpcionesComunes tiene las que comparten varios menús: comprar, programar un partido y consultar la asistencia. | controller |
| **AplicacionClub** | Carga el club desde datos/club.txt o datos/club_ejemplo.txt y pide el login | controller |
| **Entrada** | Lee y muestra textos, menús, fechas y pesos | interfacer |

| Candidato | Descripción | Estereotipo |
|---|---|---|
| **Los 9 enum** | Listas fijas de valores: niveles, modalidades, estados, motivos y tipos | information holder |

**DESCARTADOS**

| Candidato | Por qué no es una clase |
|---|---|
| **Jugador como subclase de Socio** | Un entrenador también juega partidos de práctica. Se reemplazó por PerfilJugador, que puede tener cualquier usuario. |
| **Factura** | No se guarda aparte: los totales viven en Venta. |
| **Reporte** | Un reporte es una consulta sobre datos que ya existen. Son métodos de GeneradorReportes. |
| **Rival o Institución** | De un rival solo se necesita el nombre: es un atributo de PartidoCompetencia. |
| **BloqueHorario** | Una franja es fecha, hora de inicio y hora de fin: son atributos de Reserva y de Turno. |
| **Temperatura (enum)** | Solo había dos valores, así que Bebida tiene un boolean caliente. |
| **TipoSesion (enum)** | Se sabe si la sesión tiene equipo o disciplina; un atributo aparte podía contradecirlos. |
| **HabilitacionMedica** | No se implementa en esta entrega: es una limitación (RF-E-13). |

# 4. RESPONSABILIDADES

### CÓMO LEER ESTA SECCIÓN

Una responsabilidad es algo que una clase **sabe**, **hace** o **decide**:

- **Saber:** guarda un dato y lo puede dar. Por ejemplo, `Club` sabe todos los equipos.
- **Hacer:** calcula o cambia algo cuando se lo piden. Por ejemplo, `Sancion` descuenta un partido.
- **Decidir:** responde sí o no con una regla. Por ejemplo, `Categoria` decide si una edad cabe en ella.

La pregunta de fondo es **¿quién debería hacer esto?** Nuestra respuesta casi siempre es la misma: la clase que tiene los datos que hacen falta. La última columna dice el método donde se ve cada responsabilidad.

### LAS RESPONSABILIDADES

| # | Responsabilidad | Tipo | Asignada a | En el código |
|---|---|---|---|---|
| **1** | Saber todos los usuarios, equipos, partidos internos, competencias, productos, ventas y turnos | Saber | Club | Sus 17 listas |
| **2** | Verificar login y clave | Decidir | Club y Usuario | `Club.autenticar`, `Usuario.validarPassword` |
| **3** | Calcular la edad de un usuario | Hacer | Usuario | `Usuario.calcularEdad` |
| **4** | Decidir si una edad cabe en una categoría | Decidir | Categoria | `Categoria.admiteEdad` |
| **5** | Decidir si un jugador entra a un equipo y crear la inscripción | Decidir | Equipo | `Equipo.inscribirJugador` |
| **6** | Saber en qué equipo de un deporte está un jugador | Saber | PerfilJugador | `PerfilJugador.buscarInscripcionEnDeporte` |
| **7** | Decidir si un jugador está habilitado | Decidir | InscripcionEquipo | `InscripcionEquipo.estaHabilitado` |
| **8** | Convertir tarjetas en sanciones | Hacer | InscripcionEquipo | `InscripcionEquipo.registrarTarjetas` |
| **9** | Contar los partidos que le faltan a una sanción | Hacer | Sancion | `Sancion.descontarPartido` |
| **10** | Decidir si se puede programar un partido interno | Decidir | Club | `Club.programarPartidoInterno` |
| **11** | Decidir quién puede jugar un partido | Decidir | Cada subclase de Partido | `puedeParticipar` |
| **12** | Hacer lo que toca al cerrar un partido | Hacer | Partido y sus subclases | `Partido.cerrar` y `despuesDeCerrar` |
| **13** | Inscribir un equipo en una competencia con su nómina | Decidir | Competencia | `Competencia.inscribirEquipo` |
| **14** | Registrar un resultado oficial y actualizar la tabla | Hacer | CompetenciaLiga y CompetenciaPorFases | `registrarResultado` |
| **15** | Sumar un resultado a la fila de un equipo | Hacer | FilaTabla | `FilaTabla.sumarResultado` |

| # | Responsabilidad | Tipo | Asignada a | En el código |
|---|---|---|---|---|
| **16** | Decidir qué modalidad acepta una instalación y si permite bebida caliente | Decidir | Cada subclase de Instalacion | `admite`, `prohibeBebidaCaliente` |
| **17** | Decidir si una reserva se puede hacer | Decidir | DisciplinaIndividual | `DisciplinaIndividual.reservar` |
| **18** | Decidir si dos niveles pueden jugar entre sí | Decidir | Nivel | `Nivel.puedeJugarContra` |
| **19** | Calcular el ranking después de un partido de práctica | Hacer | PartidoPractica | `PartidoPractica.cerrarYActualizarRanking` |
| **20** | Convocar, registrar asistencia y cerrar una sesión | Hacer | SesionEntrenamiento | `convocar`, `registrarAsistencia`, `cerrar` |
| **21** | Calcular el total de una venta y mover los puntos | Hacer | Venta | `Venta.cerrar` |
| **22** | Calcular el impuesto y la propina | Hacer | VentaTienda y VentaCafeteria | `calcularImpuestos`, `calcularPropina` |
| **23** | Decidir qué descuento le toca a quien compra | Decidir | CodigoDescuento | `CodigoDescuento.porcentajePara` |
| **24** | Decidir si se puede vender una bebida caliente | Decidir | Club | `Club.puedeRecibirBebidaCaliente` |
| **25** | Registrar una venta y descontar el inventario | Hacer | Club | `Club.registrarVenta` |
| **26** | Saber y mover las existencias de una ubicación | Hacer | UbicacionInventario | `retirar`, `ingresar` |
| **27** | Decidir si un cambio de turno deja la cobertura mínima | Decidir | Club | `Club.resolverCambioTurno` |
| **28** | Calcular los reportes | Hacer | GeneradorReportes | `ventasPorRubro`, `asistenciaDeJugador`, … |
| **29** | Guardar y cargar el club | Hacer | PersistenciaClub | `guardar`, `cargar` |
| **30** | Probar cada requerimiento y mostrar el resultado | Hacer | Los tres menús (MenuSocio, MenuEmpleado, MenuAdministrador) | Una opción de menú por requerimiento |

### UN EJEMPLO DE PUNTA A PUNTA

Sofía, de 22 años, se inscribe en el equipo de fútbol Mayores A desde el menú del socio (opción 2, RF-S-03). Así pasa en el código, en el orden en que se revisan las reglas.

1. `MenuSocio llama a Equipo.inscribirJugador con el perfil de Sofía, la posición, el número de camiseta y la fecha de hoy.`
2. Equipo revisa que él mismo esté activo y que Sofía esté activa (RN-35). Si no, lanza una excepción y se detiene.
3. Equipo revisa el cupo (RN-03). Si ya tiene todos sus inscritos, la operación se rechaza.
4. Equipo le pide a Usuario.calcularEdad la edad de Sofía (22) y le pregunta a Categoria.admiteEdad si esa edad cabe en Mayores A (RN-04).

5. Equipo le pregunta a PerfilJugador.buscarInscripcionEnDeporte si Sofía ya está en otro equipo de fútbol (RN-05).
6. Equipo revisa que la camiseta esté libre (tieneCamiseta, RN-06) y que la posición no esté vacía (RN-07).
7. Solo si todo pasó, Equipo crea la InscripcionEquipo, la guarda con agregarInscripcion, se la pasa a Sofía con PerfilJugador.agregarInscripcionEquipo y la devuelve al menú.

Si Sofía intenta lo mismo en un segundo equipo de fútbol, el paso 5 encuentra la inscripción anterior y la operación se rechaza con JugadorNoHabilitadoException (RN-05). Como la excepción sale antes del paso 7, no se crea ni se guarda nada.

Ninguna clase hace todo: cada una responde lo que sabe. Club ni siquiera participa, porque nada de esto cruza partes del modelo.

El diagrama 1 de la sección 5 muestra este mismo caso como diagrama de secuencia.

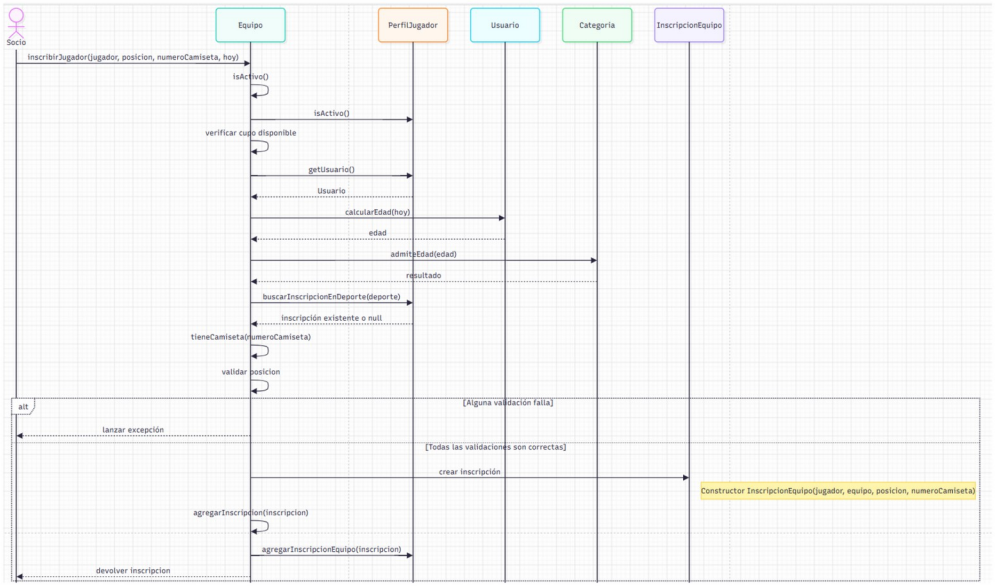
### POR QUÉ ASÍ

- **Cada clase decide con lo que sabe.** Equipo sabe su cupo y sus camisetas, Categoria su rango de edad, Instalacion qué modalidad acepta. Por eso esas reglas viven ahí y no en Club.
- **Club solo decide lo que cruza partes del modelo.** La bebida caliente (tienda e individuales), la cobertura de turnos (todos los empleados a la vez) y el partido interno (dos equipos) no le pertenecen a ninguna otra clase.
- **Que un jugador esté habilitado no se guarda: se calcula.** InscripcionEquipo lo responde mirando si tiene sanciones sin cumplir. Así nunca queda desactualizado.
- **Venta calcula su total sin preguntar de qué tipo es.** Le pide el impuesto y los cargos a su subclase.
- **Ninguna clase del modelo sabe de archivos ni imprime.** Eso es de la persistencia y de la interfaz.

## 5. COLABORACIONES

Diagrama 1. Inscripción de un jugador en un equipo

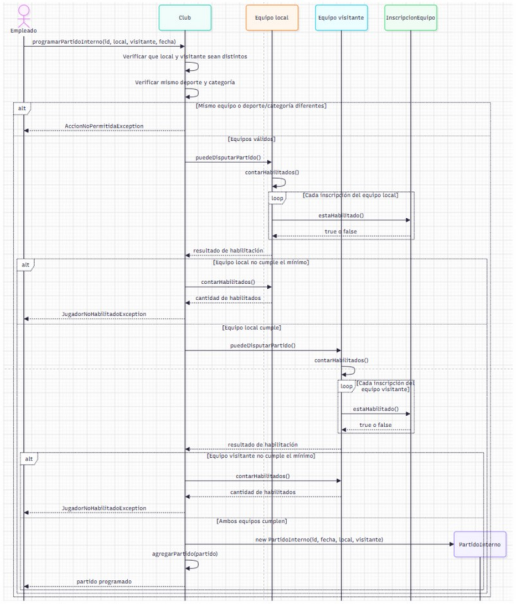
Objetivo: mostrar cómo se validan las condiciones antes de crear la inscripción.




Por qué es crítico: es el proceso que conecta al jugador con un equipo y debe impedir inscripciones que incumplan las reglas de edad, cupo, deporte o número de camiseta.

Diagrama 2. Programación y cierre de un partido interno

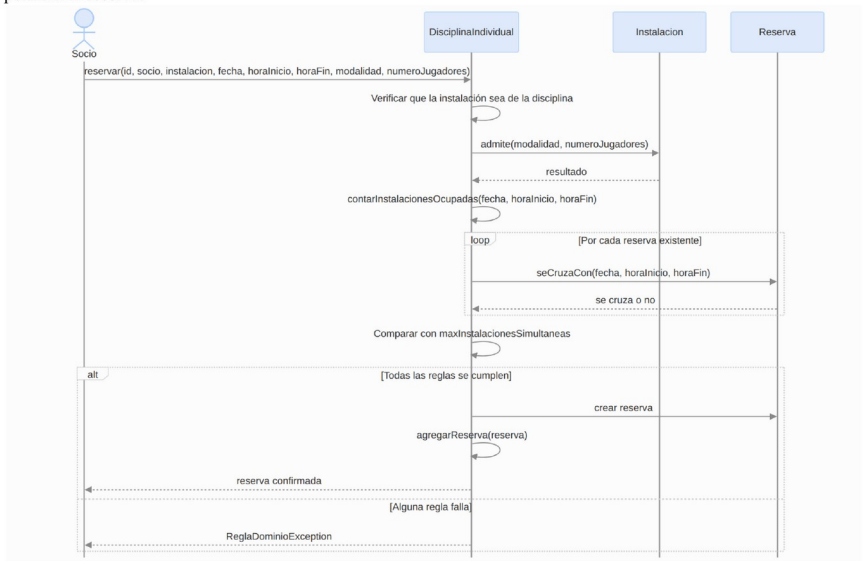
Objetivo: mostrar la coordinación entre el club, los equipos, el partido y las inscripciones de los jugadores.




Por qué es crítico: intervienen varios objetos y el cierre de un partido puede modificar sanciones y habilitaciones de los jugadores. La programación se muestra en el mismo diagrama para mantener el escenario legible.

Diagrama 3. Reserva de una instalación

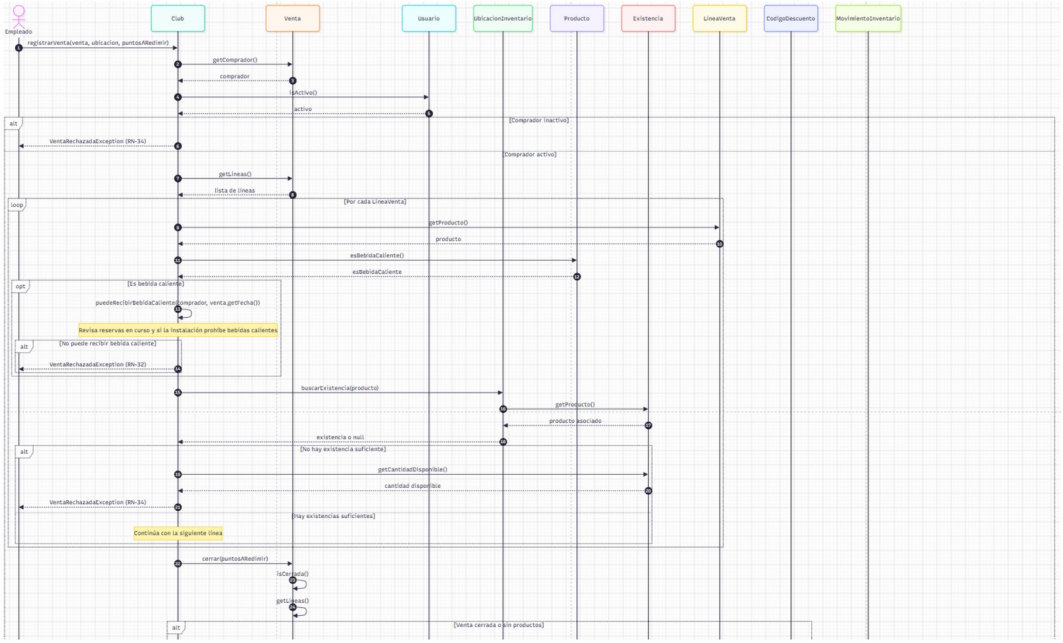
Objetivo: mostrar cómo se valida que la modalidad, el número de jugadores, el bloque horario y el máximo de instalaciones simultáneas permian la reserva.

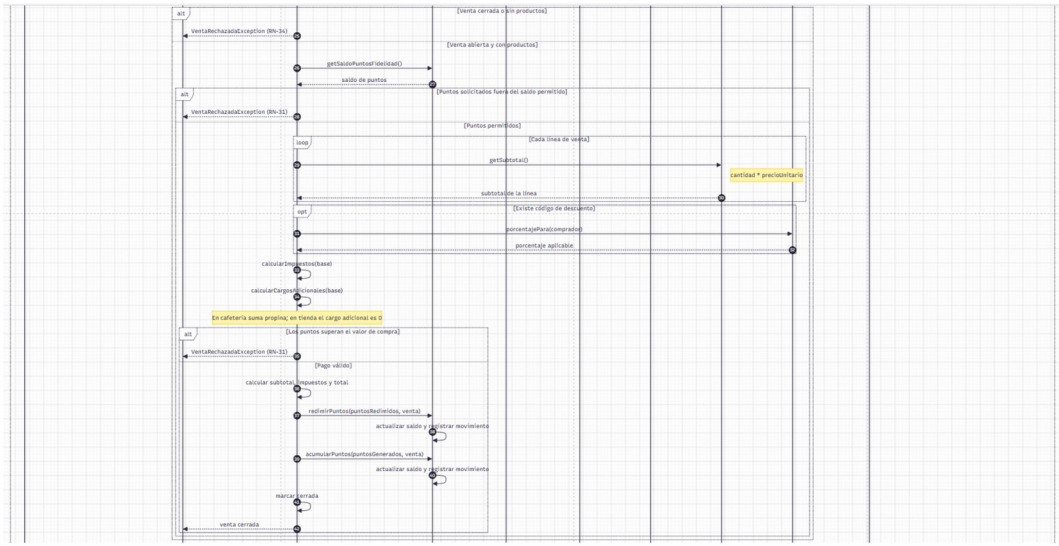


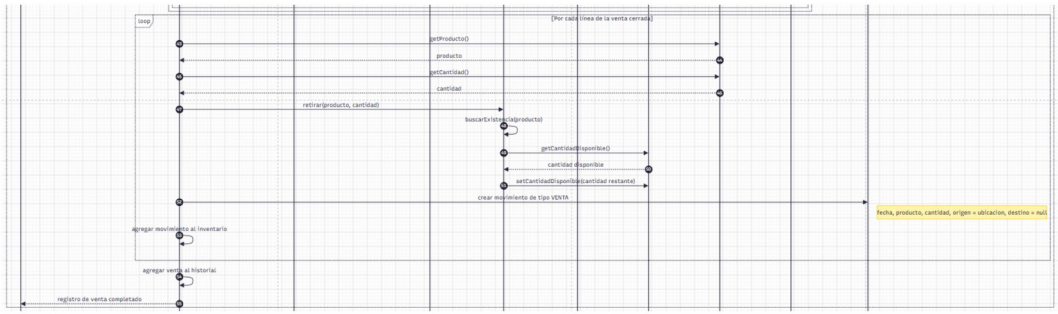

Porque es crítico: es la regla que evita reservas que superen la capacidad de una instalación o el cupo de la disciplina (RF-S-08). Cada tipo de instalación (CanchaTenias, CanchaPadel y MesaTenisMesa) responde a su manera, sin que DisciplinaIndividual pregunte de qué tipo es.

**Diagrama 4. Venta en la cafetería**

Objetivo: mostrar cómo se valida una venta (bebida caliente y existencias) y como se cierran el impuesto, la propina, los puntos de fidelidad y el inventario.



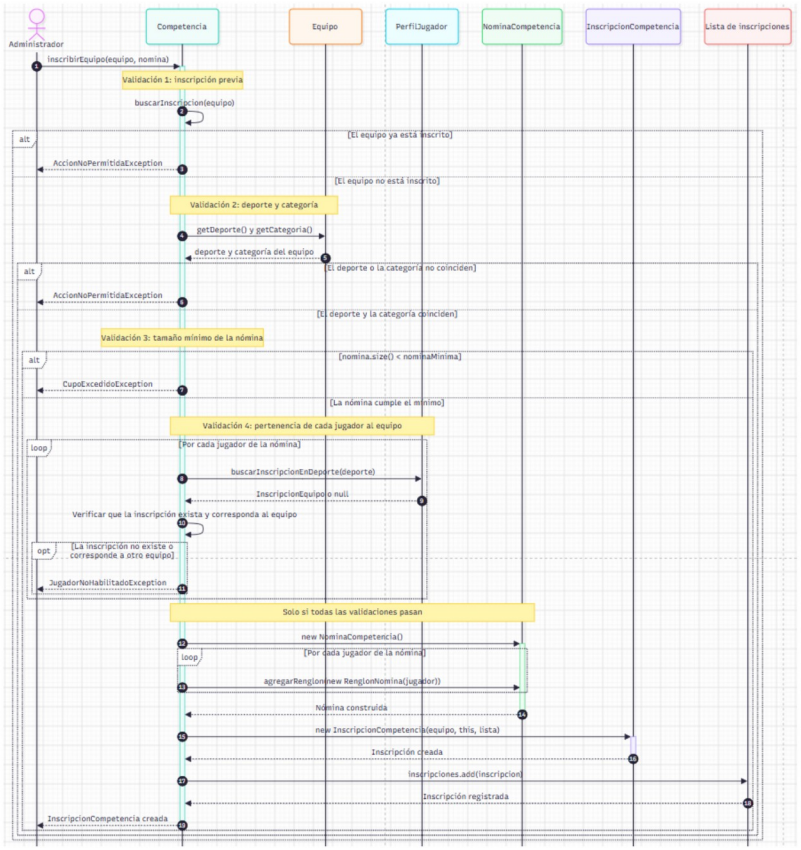

alt
[Venta cerrada o sin productos]
VentaRechazadaException (RN-34)
[Venta abierta y con productos]
getSaldoPuntosFidelidad()
saldo de puntos
alt
[Puntos solicitados fuera del saldo permitido]
VentaRechazadaException (RN-31)
[Puntos permitidos]
loop
[Cada línea de venta]
getSubtotal()
cantidad * precioUnitario
subtotal de la línea
opt
[Existe código de descuento]
porcentajePara(comprador)
porcentaje aplicable
calcularImpuestos(base)
calcularCargosAdicionales(base)
En cafetería suma propina; en tienda el cargo adicional es 0
alt
[Los puntos superan el valor de compra]
VentaRechazadaException (RN-31)
[Pago válido]
calcular subtotal, impuestos y total
redimirPuntos(puntosRedimidos, venta)
actualizar saldo y registrar movimiento
acumularPuntos(puntosGenerados, venta)
actualizar saldo y registrar movimiento
marcar cerrada
venta cerrada




Por qué es crítico: conecta ventas, productos, inventario y fidelización. También es un escenario importante para verificar que un rechazo no deje cambios parciales.

**Diagrama 5. Inscripción de un equipo a una competencia**

objetivo: mostrar cómo se validan el deporte, la categoría, la nómina mínima y la inscripción de los jugadores al equipo antes de registrar la inscripción.




Porque es crítico: solo los jugadores incluidos en la nómina pueden disputar los partidos oficiales, y una inscripción incorrecta puede afectar los partidos, la tabla de posiciones y las estadísticas de la competencia (RF-E-08 y RF-A-03).

## 6. CLASES: ALTO NIVEL

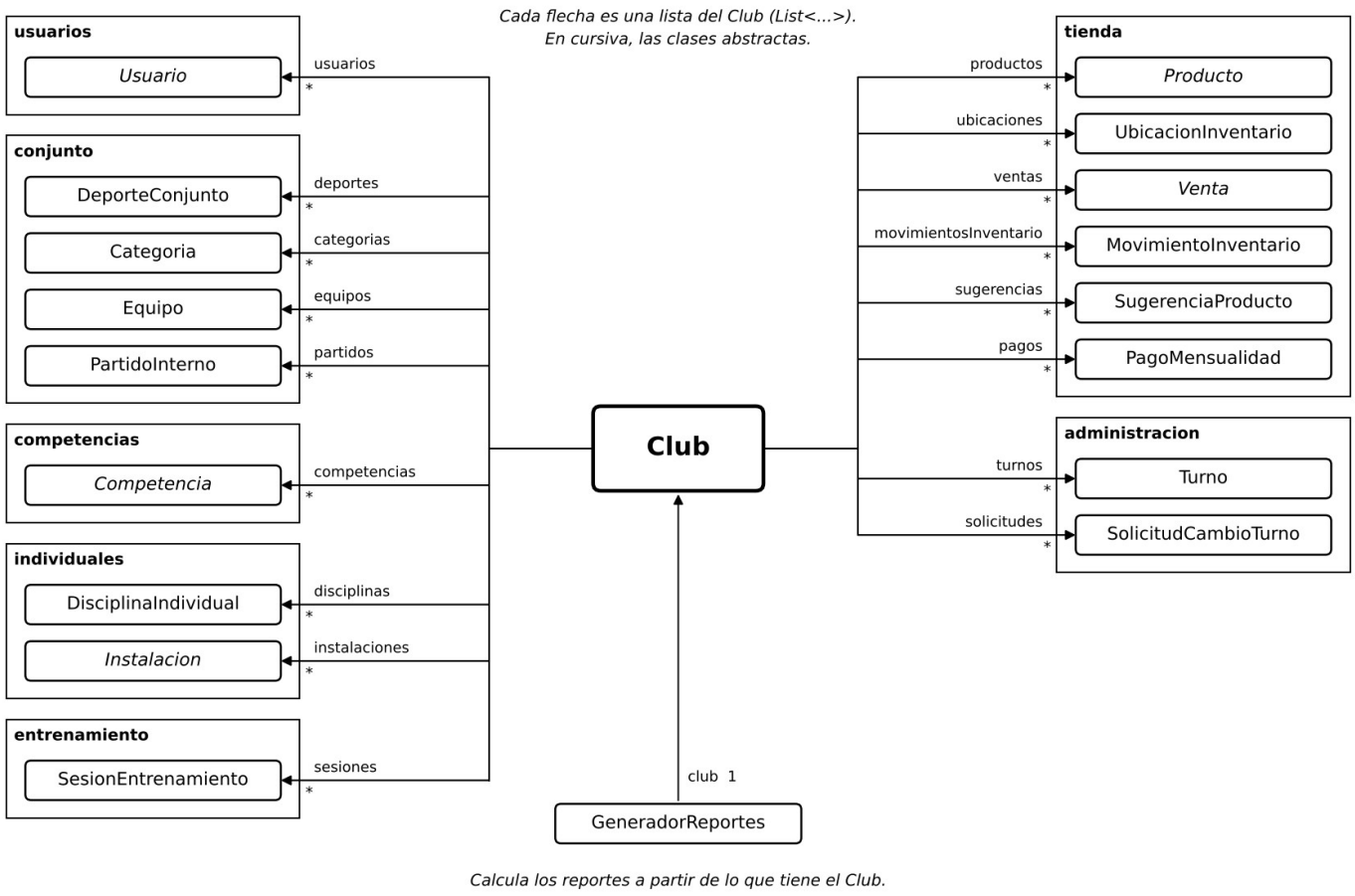
Este es el diagrama **(b)** que pide el enunciado: todas las clases y sus relaciones, sin atributos ni métodos. Sirve para ver la estructura de un vistazo. Para que se pueda leer, va en seis partes: primero el Club, que une todo, y después una parte por cada área del club.

### CÓMO LEER LOS DIAGRAMAS

| Lo que se ve | Qué quiere decir |
|---|---|
| **Flecha →** | Asociación: la clase de donde sale tiene un atributo de la clase a donde llega. |
| **Línea sin flecha** | Asociación en las dos direcciones: cada clase conoce a la otra. |
| **Triángulo vacío** | Herencia: la clase de abajo extiende a la de arriba. |
| **1 y *** | Multiplicidad: 1 es un solo objeto, * es una lista. |
| **Nombre en cursiva** | Clase abstracta: nunca se crea directamente, siempre se crea una de sus subclases. |
| **Clase fuera del recuadro** | Es de otro paquete. Aparece solo para mostrar con quién se relaciona. |

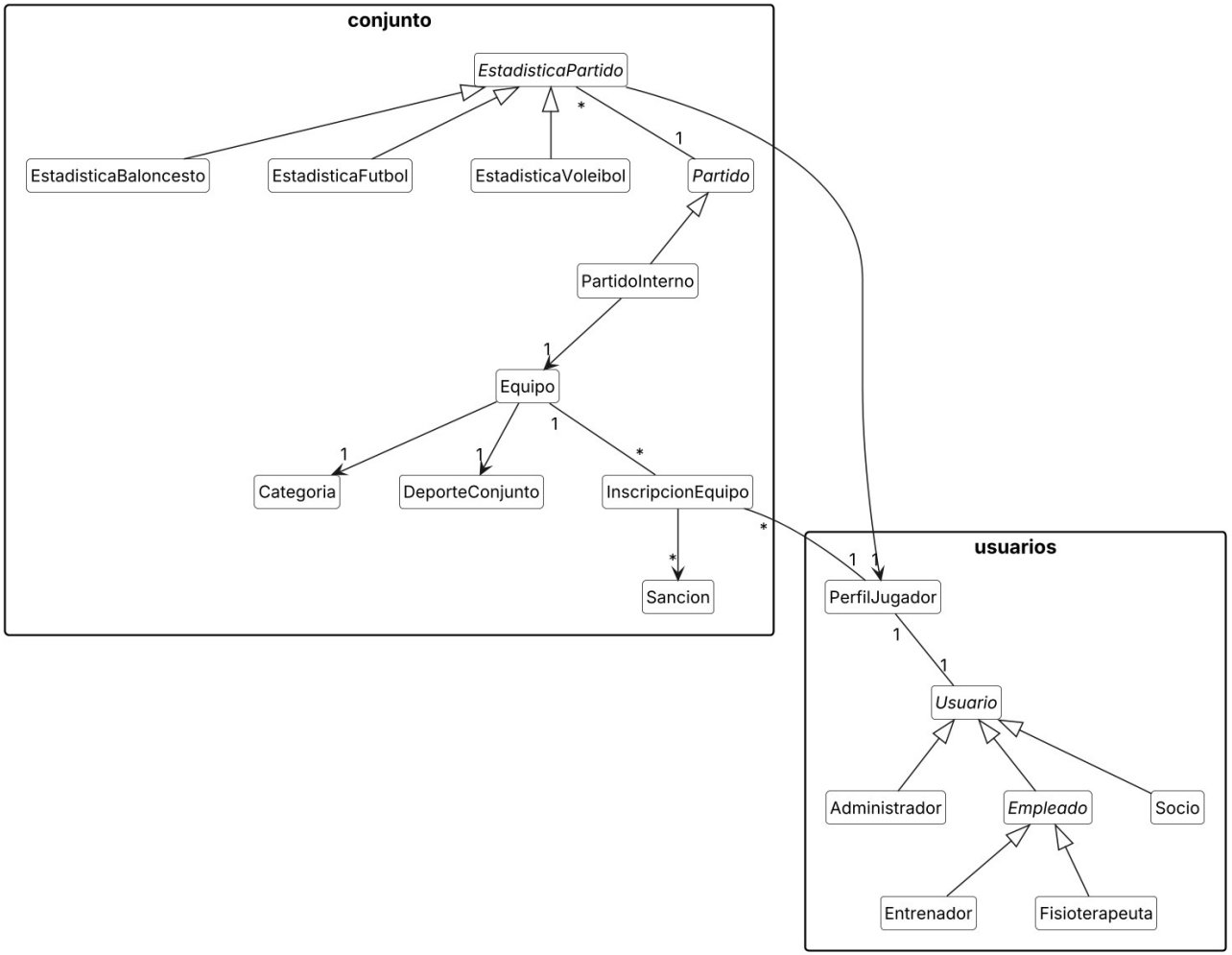
## EL CLUB, LA RAÍZ DEL MODELO

Club no es una superclase: es la raíz del estado del sistema. Mantiene listas de las entidades principales, como usuarios, equipos, competencias, disciplinas, ventas y turnos. Otras clases se relacionan con esas entidades según las responsabilidades del modelo.



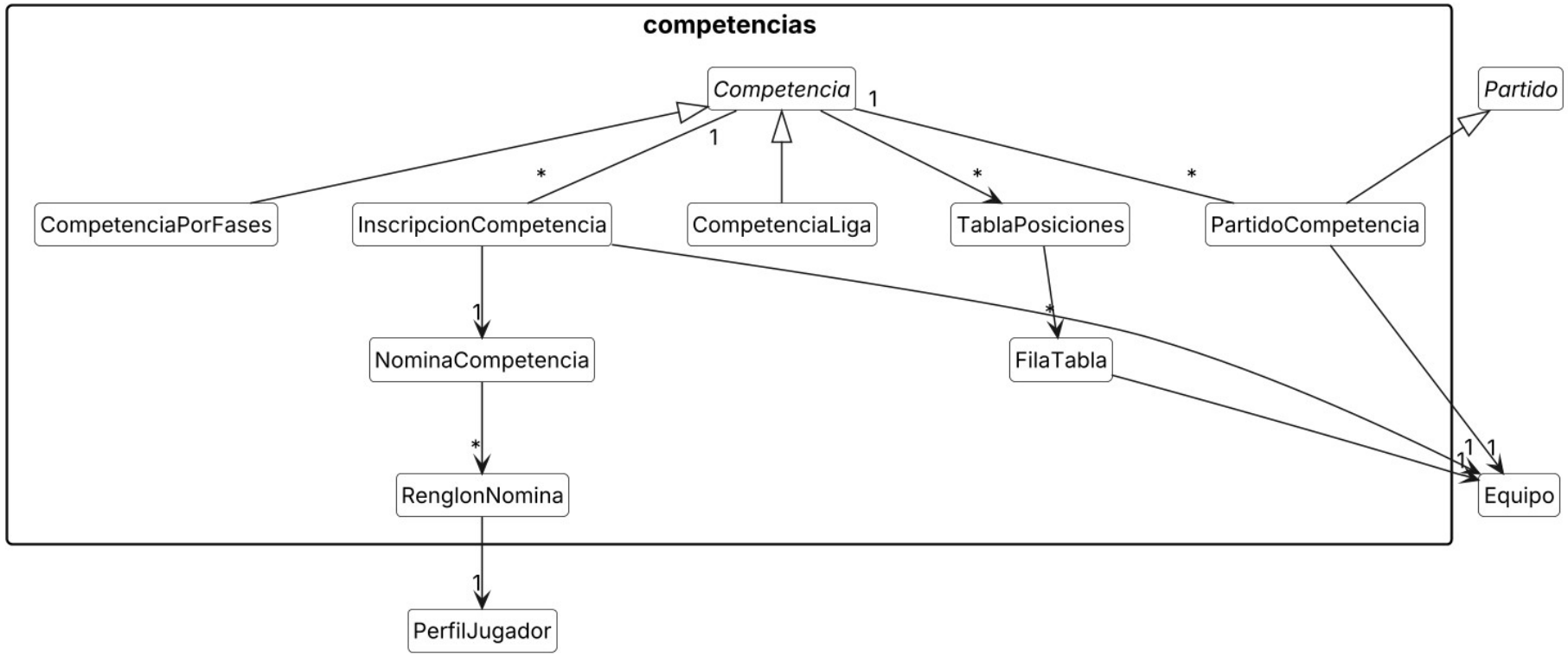
**USUARIOS Y DEPORTES DE CONJUNTO**

El diagrama muestra las jerarquías de herencia de Usuario y EstadisticaPartido, además de las relaciones entre equipos, partidos, jugadores e inscripciones. InscripcionEquipo conecta a un jugador con su equipo y registra su posición, número de camiseta, amarillas acumuladas y sanciones.



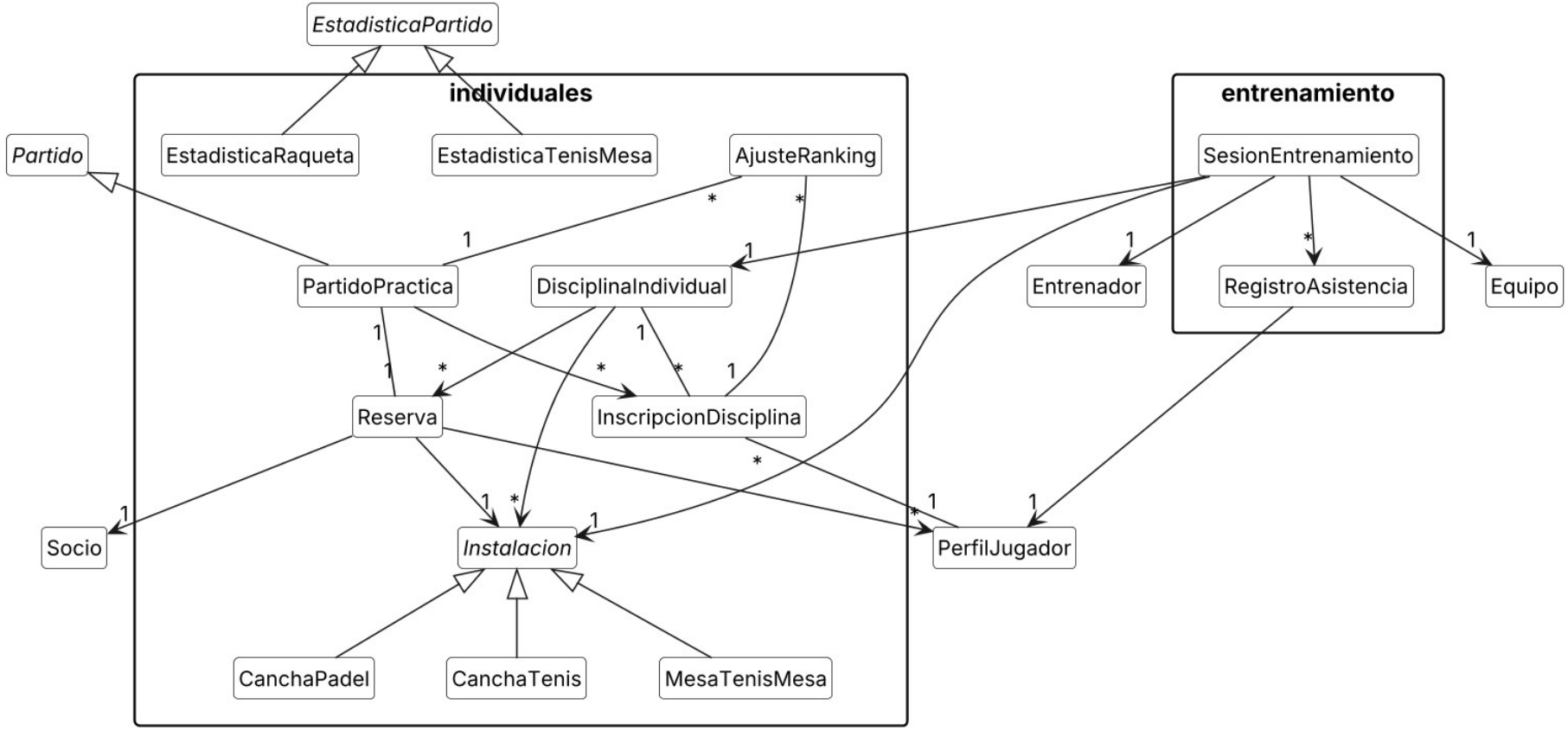
**COMPETENCIAS**

Los dos formatos de torneo heredan de Competencia. PartidoCompetencia hereda de Partido, el mismo de los partidos internos.



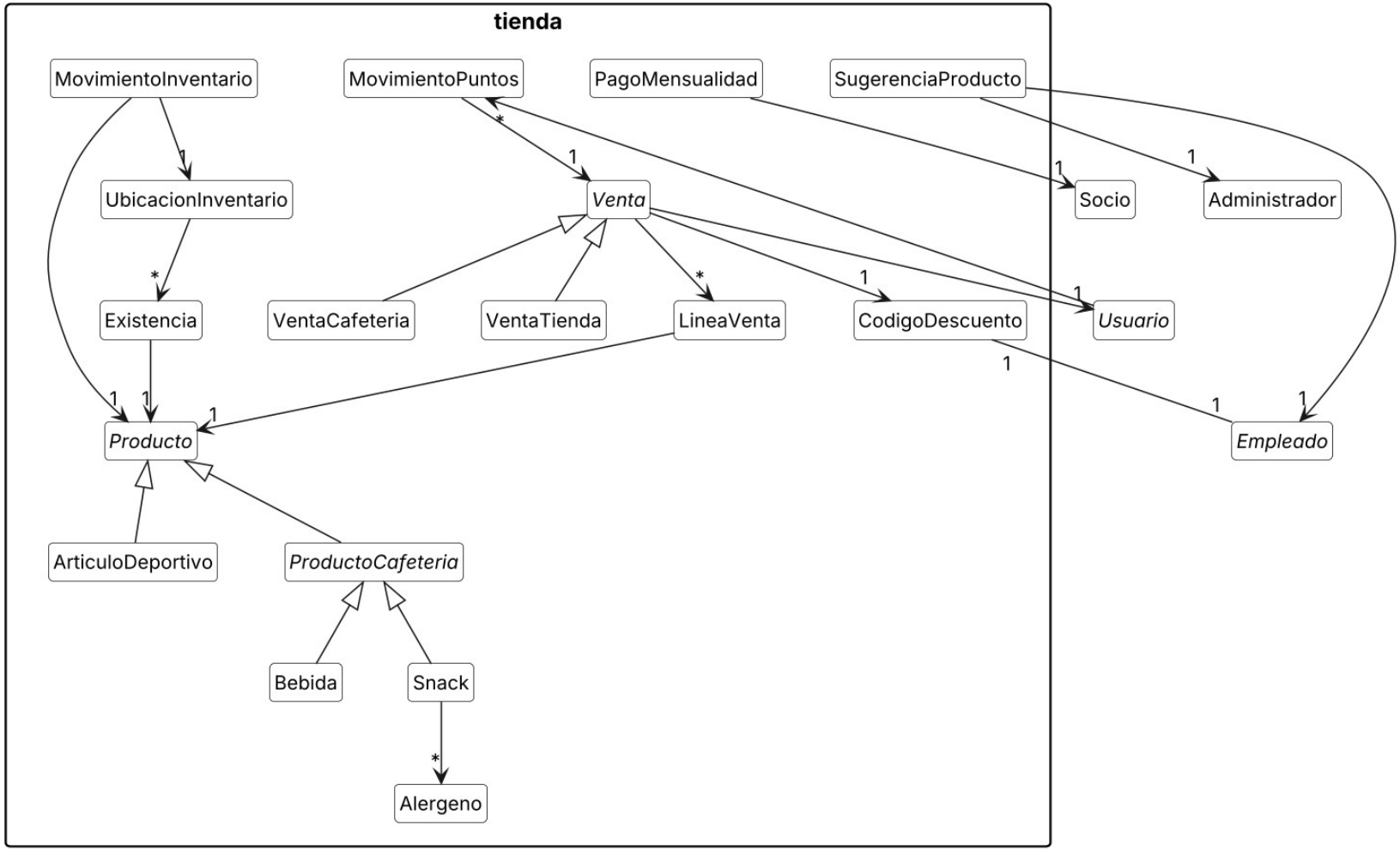
**DEPORTES INDIVIDUALES Y ENTRENAMIENTO**

Instalacion es una clase abstracta cuyas subclases definen las modalidades y capacidades admitidas. SesionEntrenamiento se asocia con un equipo o con una disciplina individual y registra la asistencia de sus participantes.



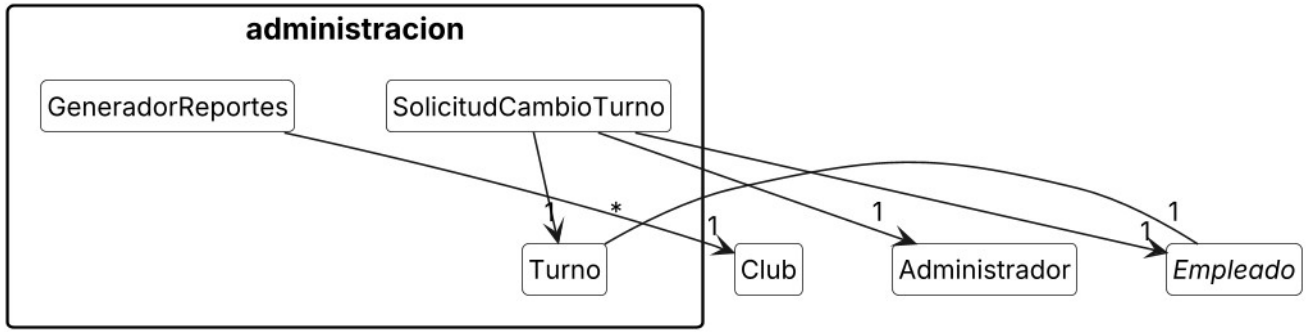
**TIENDA**

Dos jerarquías: los productos (con ProductoCafeteria en medio) y las ventas (tienda y cafetería). Existencia une un producto con el lugar donde está.



**ADMINISTRACIÓN**

Los turnos del personal, las solicitudes de cambio y los reportes, que se calculan a partir del Club.



# 7. CLASES: DETALLADO

Este es el diagrama **(a)** que pide el enunciado: las mismas clases de la sección 6, pero ahora con sus **atributos** y sus **métodos con firma completa** (parámetros y tipo de retorno). Mientras el alto nivel muestra cómo está organizado el sistema, el detallado muestra qué tiene y qué sabe hacer cada clase. Es lo más cercano al código.

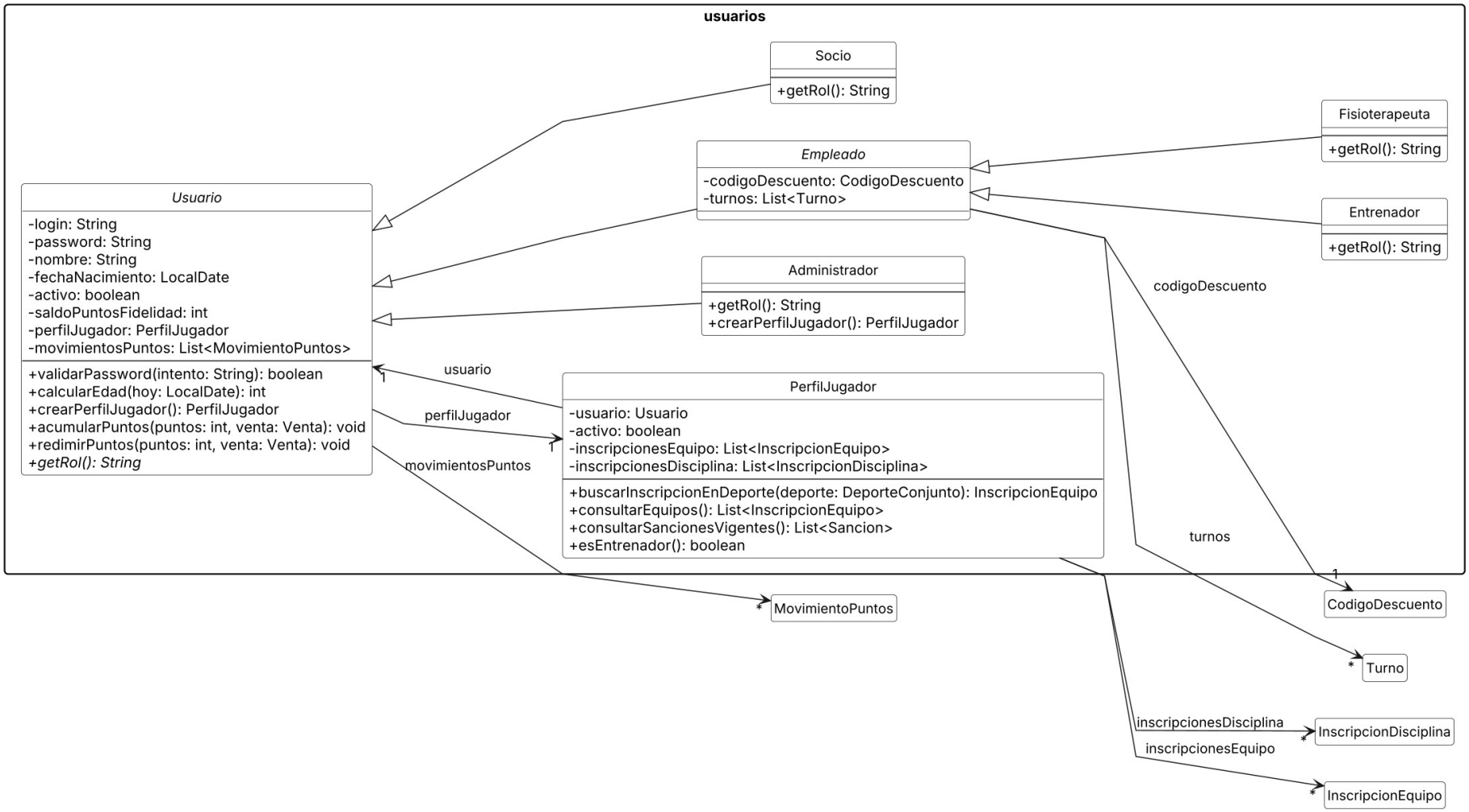
### CÓMO LEER LOS DIAGRAMAS

| Lo que se ve | Qué quiere decir |
|---|---|
| **Primer bloque de la caja** | Los atributos, con su tipo. Por ejemplo `-cupoMaximo: int`. |
| **Segundo bloque de la caja** | Los métodos: `nombre(parametro: Tipo): TipoQueDevuelve`. |
| **- + #** | Privado, público y protegido. Todos los atributos son privados. |
| **Método en cursiva** | Método abstracto: cada subclase lo escribe a su manera. |
| **{static}** | Constante de la clase, por ejemplo `TASA_IVA = 0.19`. |
| **Clase sin bloques** | Es de otra parte del modelo. Su detalle está en su propio diagrama. |

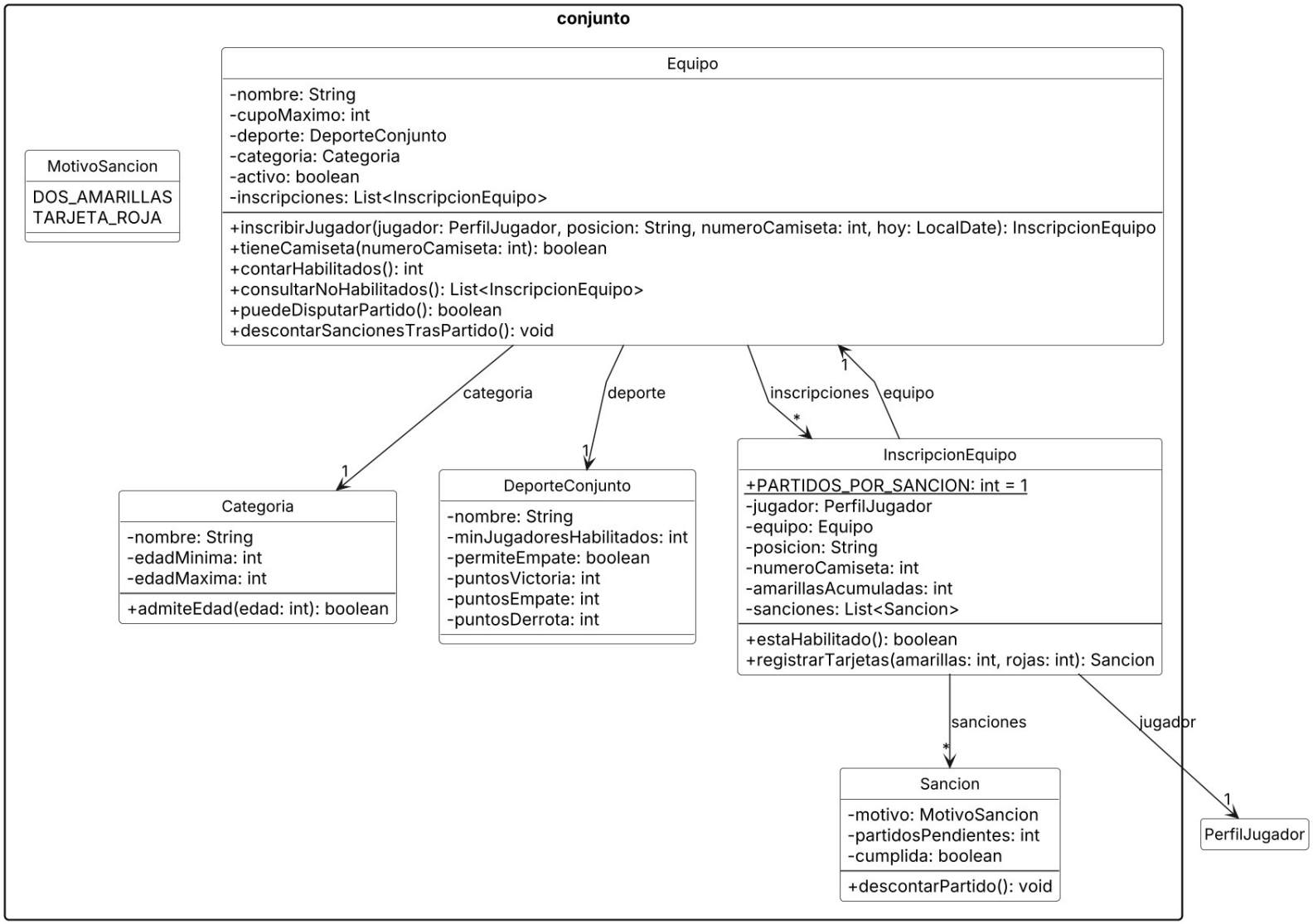
**Lo que no se dibuja:** todos los atributos tienen su getter (`getX`, o `isX` si es boolean) y cada lista tiene su método `agregarX`. Los atributos que se pueden cambiar tienen setter. Esos métodos no aparecen, para que los diagramas se puedan leer.

Los diagramas se generaron leyendo el código, así que los nombres y las firmas son los mismos del código. La descripción de cada clase está en la sección 3.

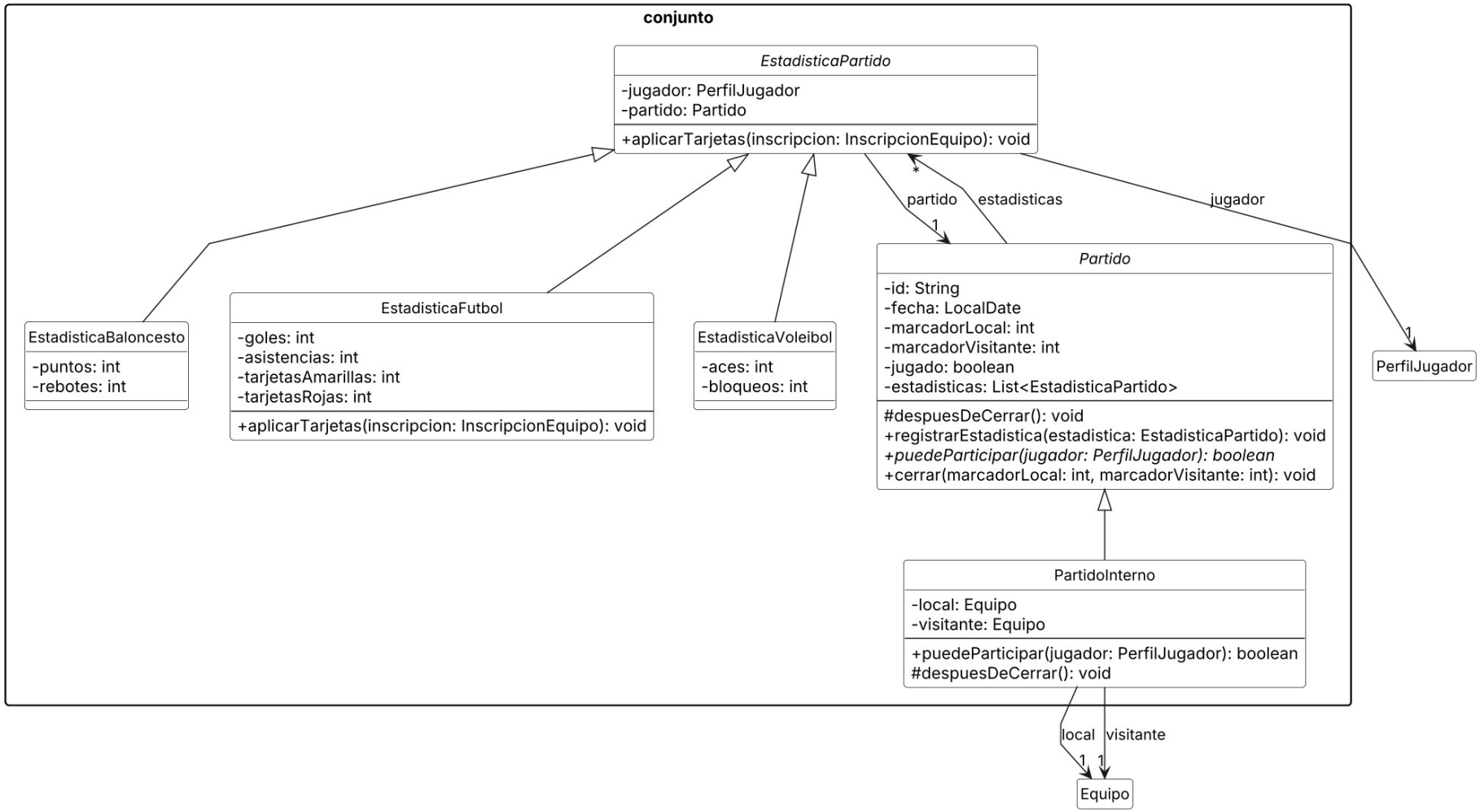
## USUARIOS


usuarios
Socio
+getRol(): String
Fisioterapeuta
+getRol(): String
Empleado
-codigoDescuento: CodigoDescuento
-turnos: List<Turno>
Entrenador
+getRol(): String
Usuario
-login: String
-password: String
-nombre: String
-fechaNacimiento: LocalDate
-activo: boolean
-saldoPuntosFidelidad: int
-perfilJugador: PerfilJugador
-movimientosPuntos: List<MovimientoPuntos>
+validarPassword(intento: String): boolean
+calcularEdad(hoy: LocalDate): int
+crearPerfilJugador(): PerfilJugador
+acumularPuntos(puntos: int, venta: Venta): void
+redimirPuntos(puntos: int, venta: Venta): void
+getRol(): String
Administrador
+getRol(): String
+crearPerfilJugador(): PerfilJugador
codigoDescuento
usuario
1
perfilJugador
1
movimientosPuntos
PerfilJugador
-usuario: Usuario
-activo: boolean
-inscripcionesEquipo: List<InscripcionEquipo>
-inscripcionesDisciplina: List<InscripcionDisciplina>
+buscarInscripcionEnDeporte(deporte: DeporteConjunto): InscripcionEquipo
+consultarEquipos(): List<InscripcionEquipo>
+consultarSancionesVigentes(): List<Sancion>
+esEntrenador(): boolean
turnos
1
*
MovimientoPuntos
CodigoDescuento
*
Turno
inscripcionesDisciplina
*
InscripcionDisciplina
inscripcionesEquipo
*
InscripcionEquipo

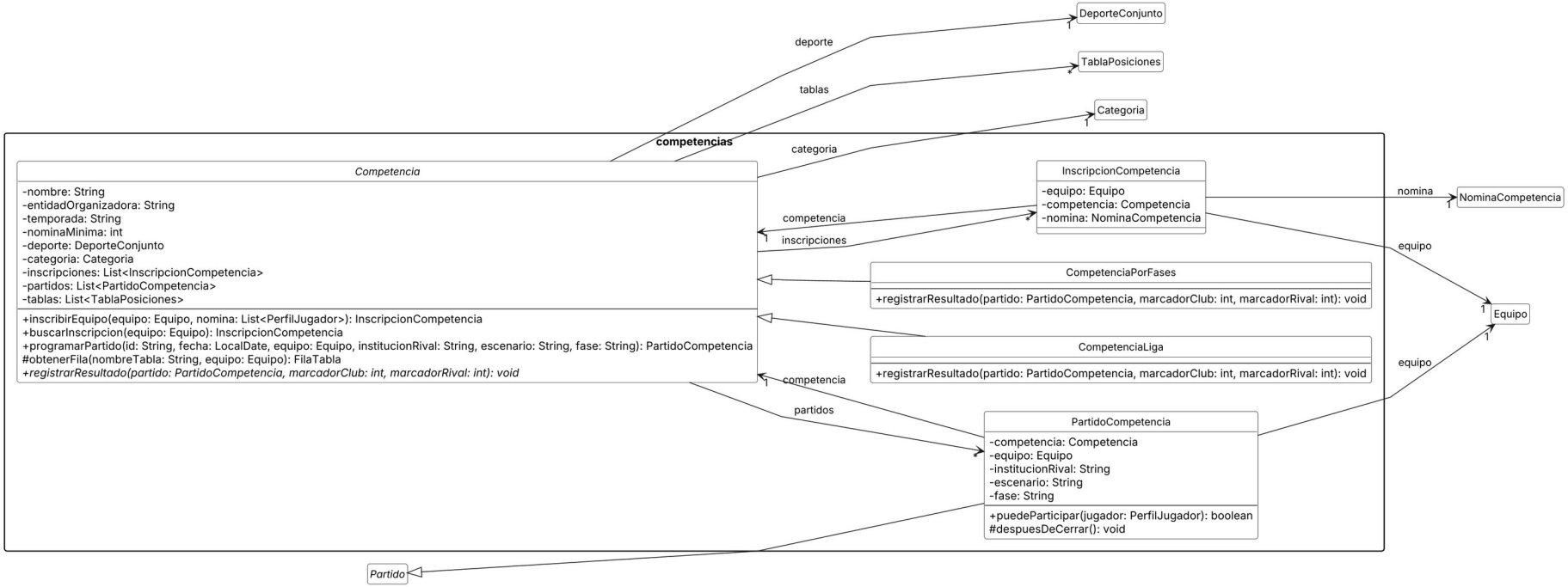
## CONJUNTO: EQUIPOS, INSCRIPCIONES Y SANCIONES



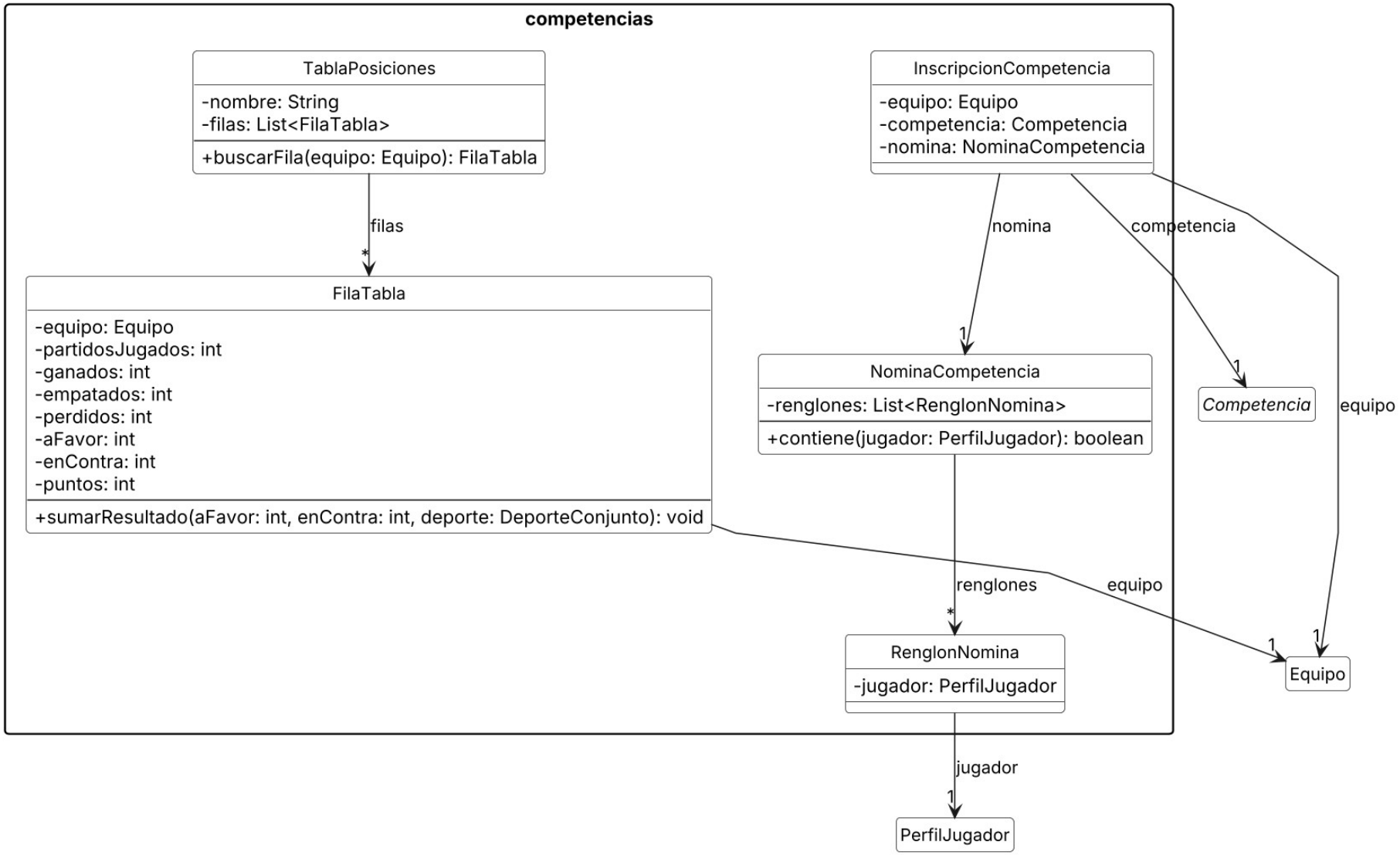
## CONJUNTO: PARTIDOS Y ESTADÍSTICAS



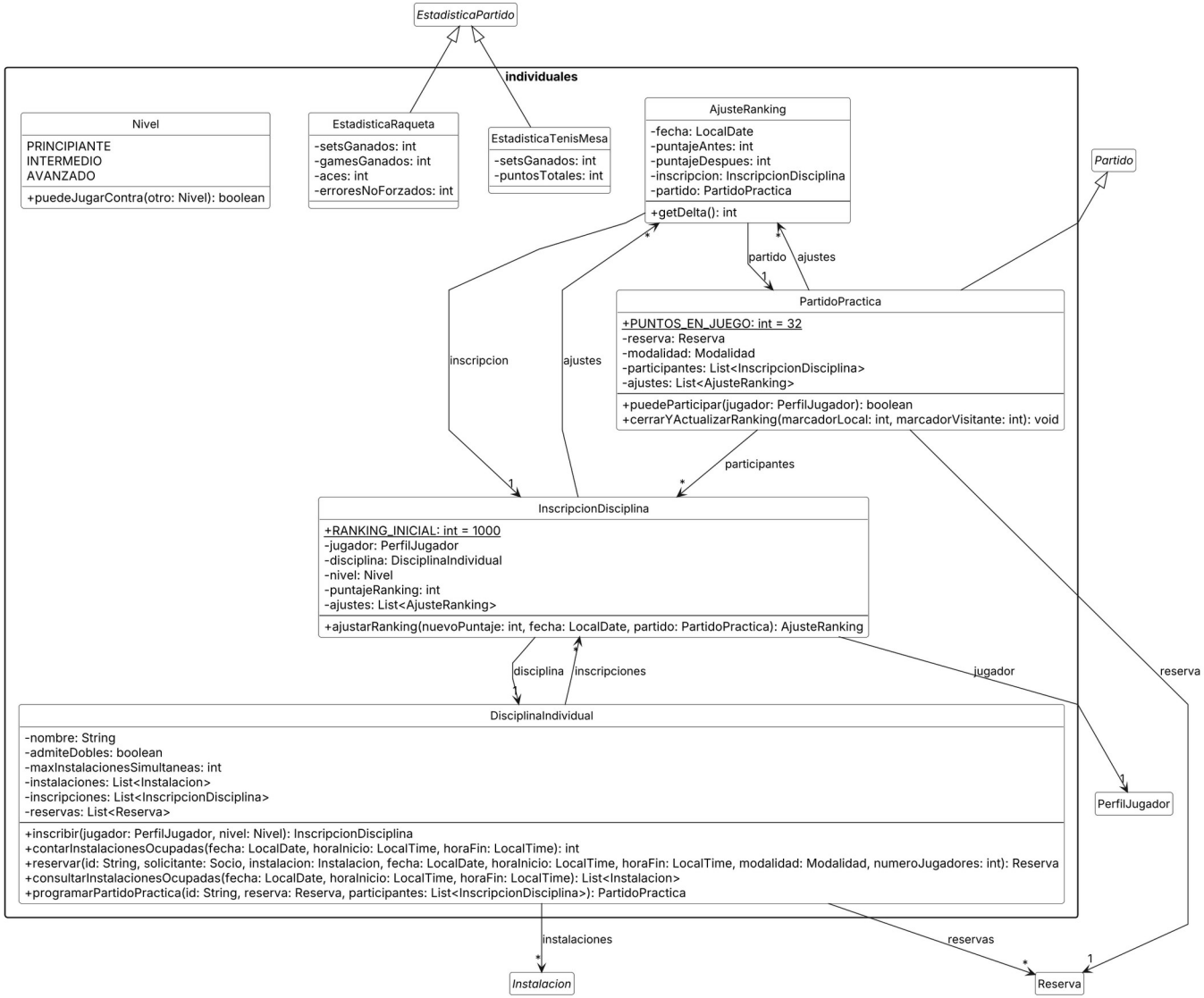
## COMPETENCIAS: TORNEOS, PARTIDOS E INSCRIPCIONES



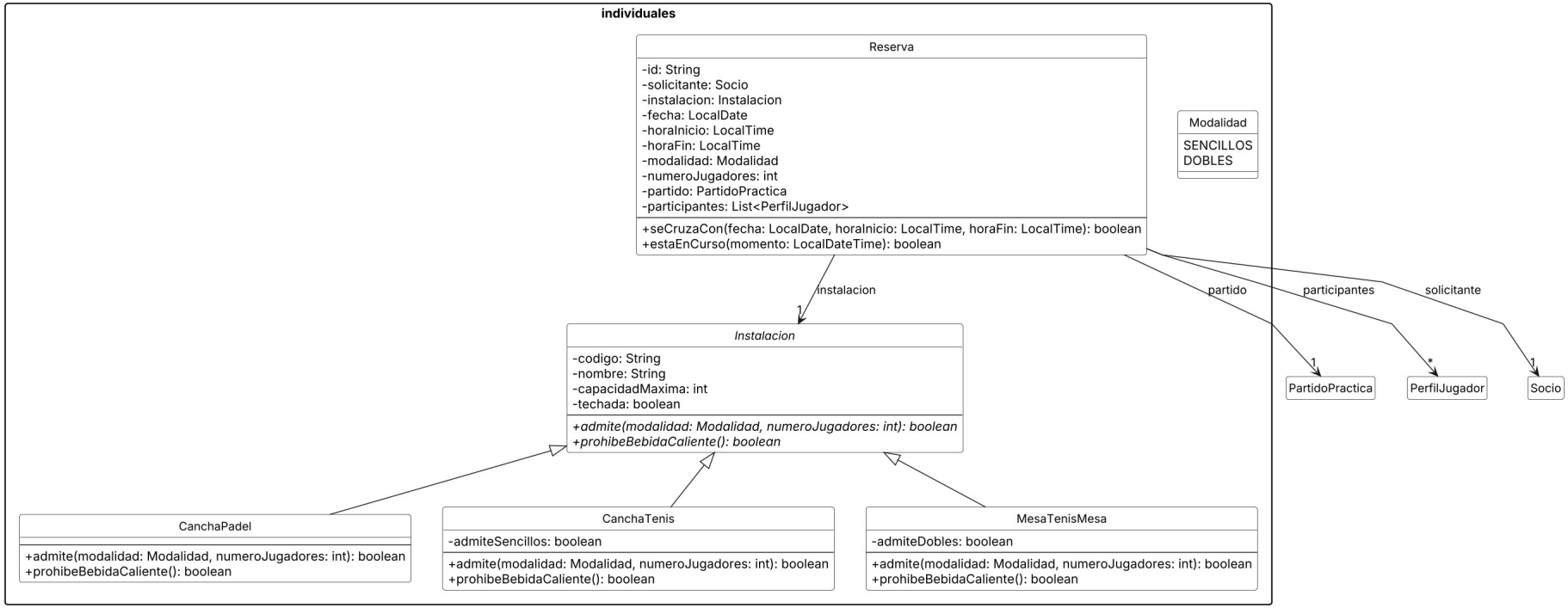
**COMPETENCIAS: NÓMINAS Y TABLAS**



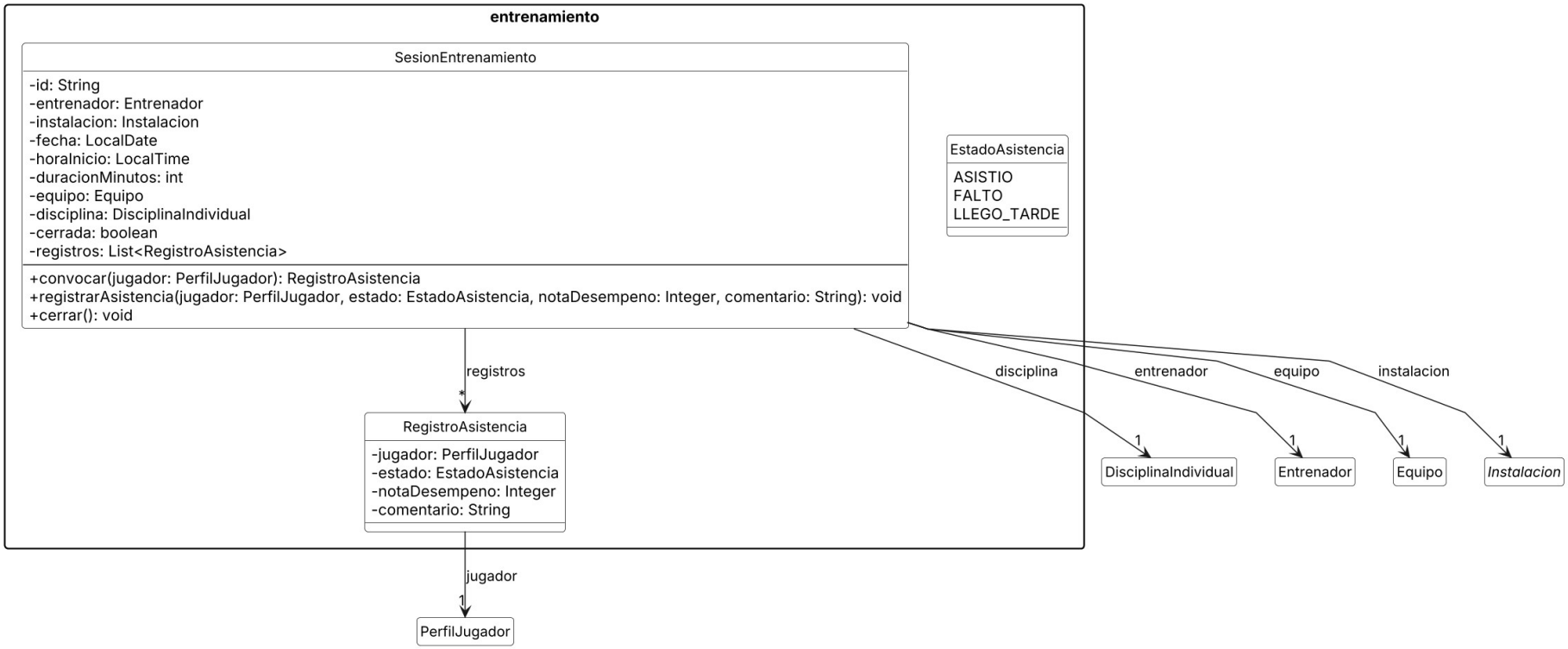
## INDIVIDUALES: DISCIPLINAS, NIVELES Y RANKING



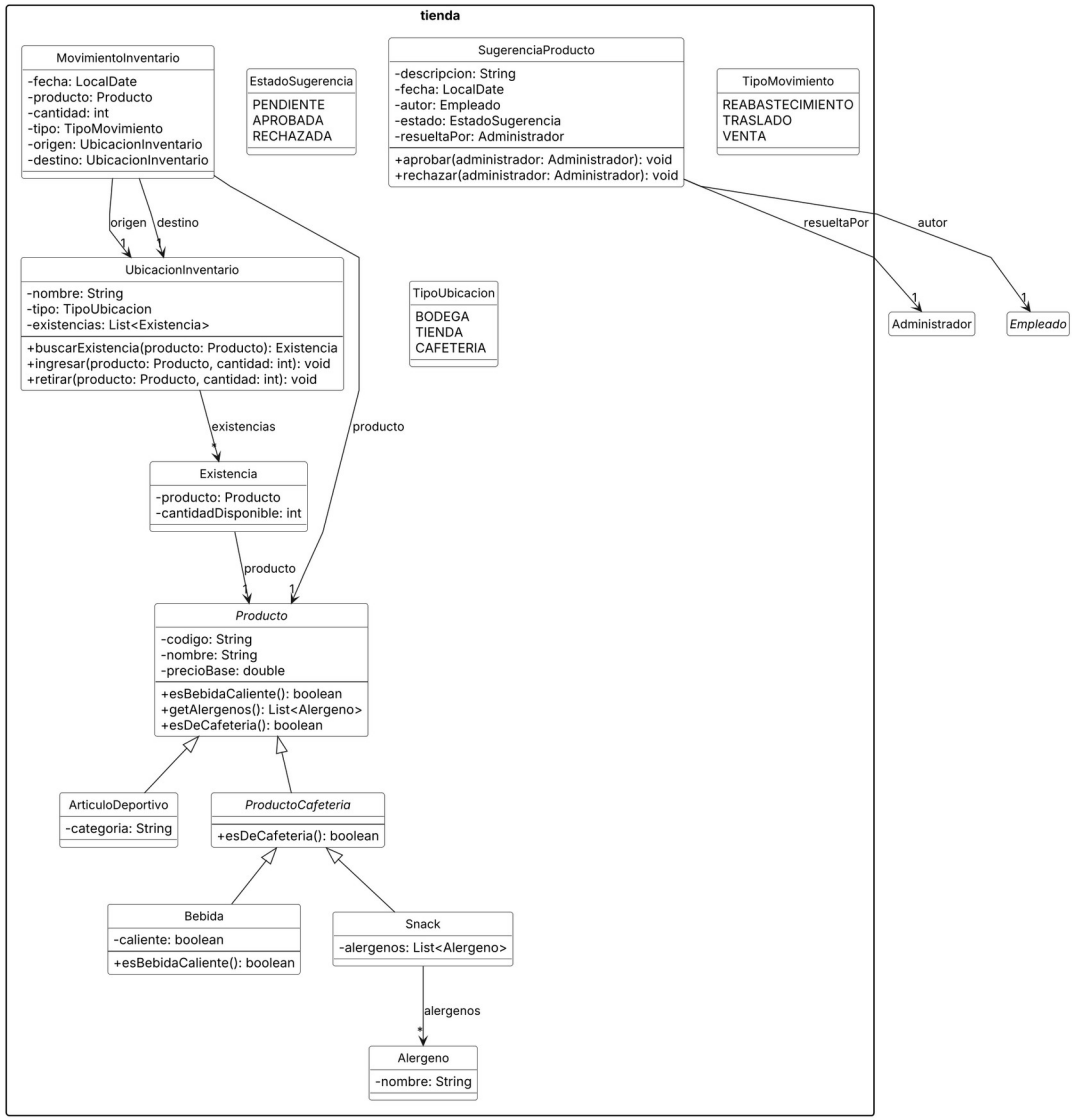
## INDIVIDUALES: INSTALACIONES Y RESERVAS



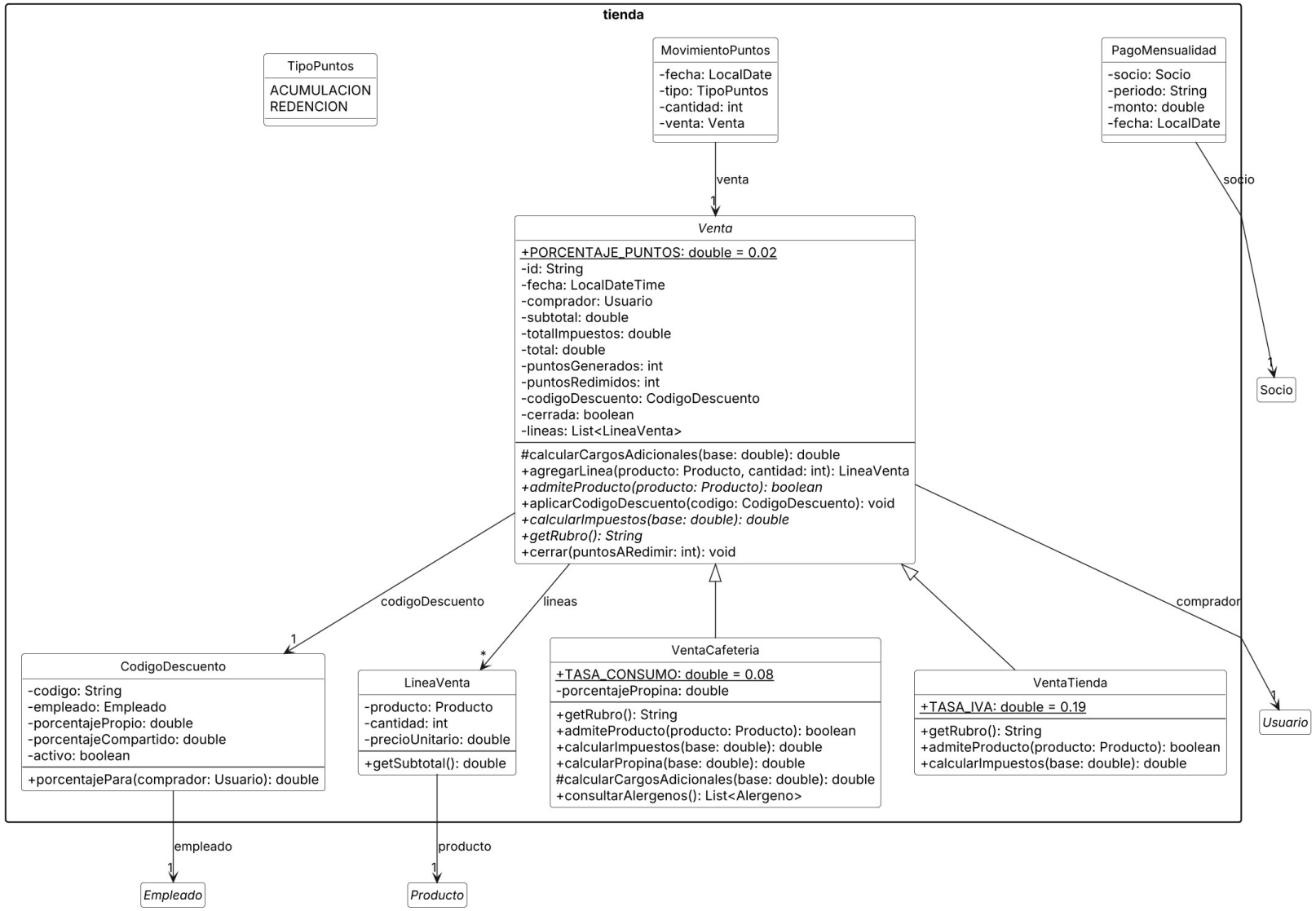
## ENTRENAMIENTO



# TIENDA: PRODUCTOS E INVENTARIO



## TIENDA: VENTAS, DESCUENTOS Y PUNTOS

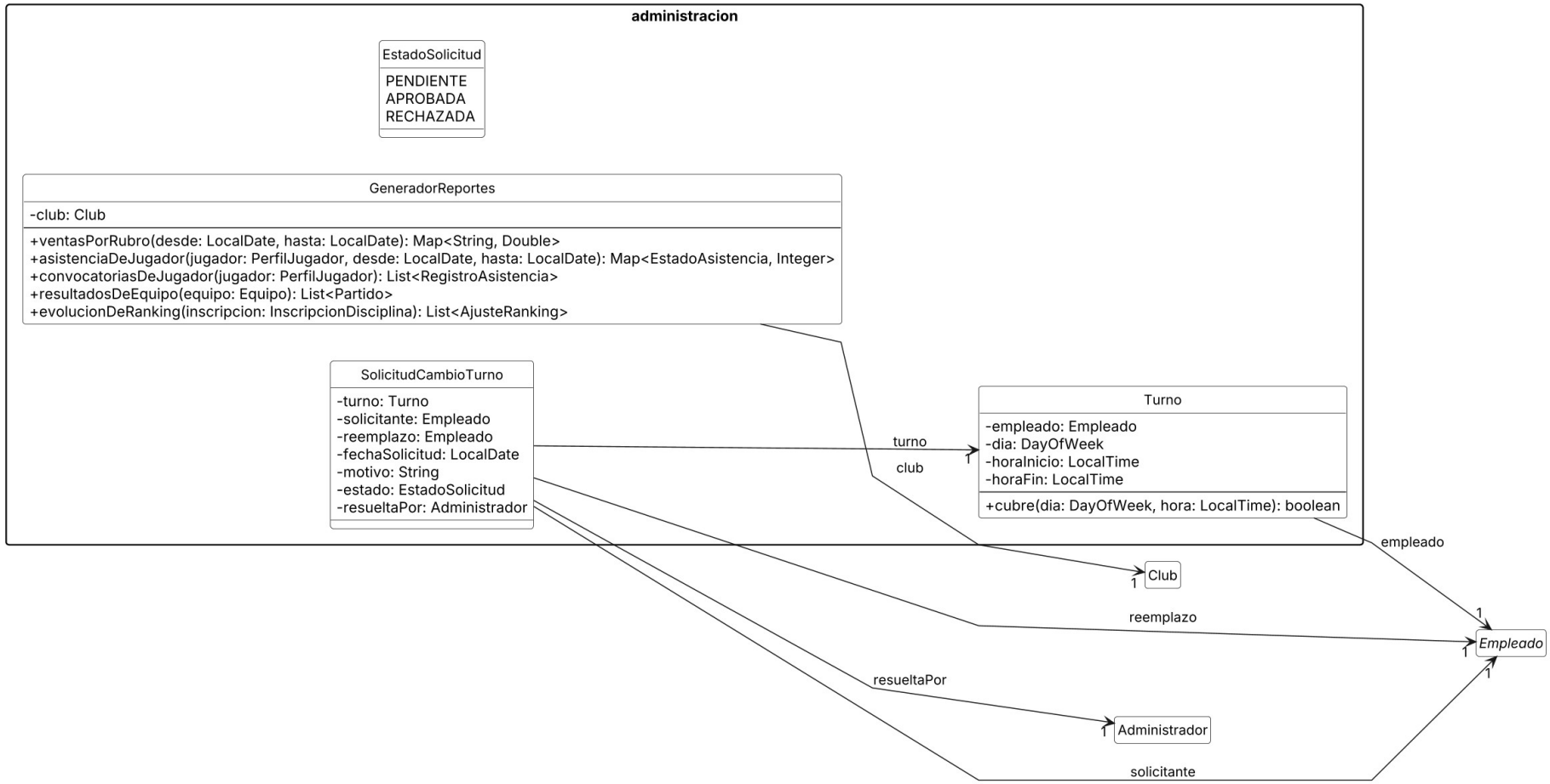


## ADMINISTRACION: CLUB

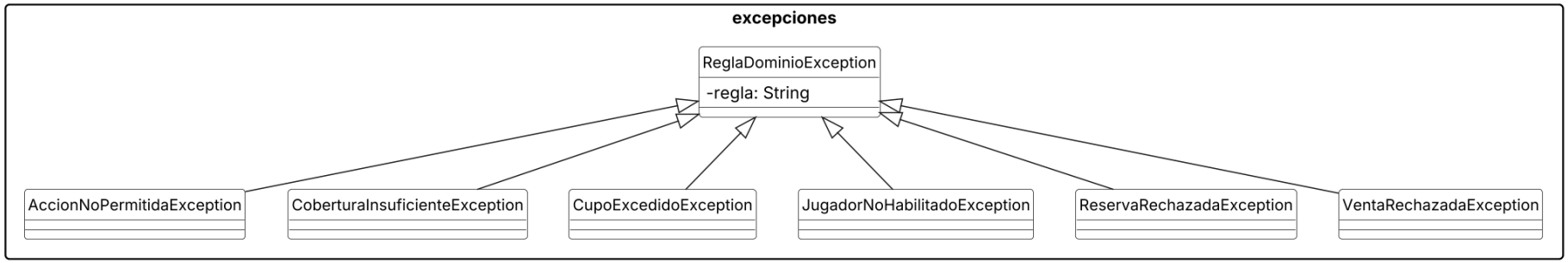
**administracion**

| Club |
| --- |
| -nombre: String<br>-horaApertura: LocalTime<br>-horaCierre: LocalTime<br>-minEntrenadoresPorFranja: int<br>-minFisioterapeutasPorFranja: int<br>-usuarios: List<Usuario><br>-deportes: List<DeporteConjunto><br>-categorias: List<Categoria><br>-equipos: List<Equipo><br>-partidos: List<PartidoInterno><br>-competencias: List<Competencia><br>-disciplinas: List<DisciplinaIndividual><br>-instalaciones: List<Instalacion><br>-sesiones: List<SesionEntrenamiento><br>-productos: List<Producto><br>-ubicaciones: List<UbicacionInventario><br>-ventas: List<Venta><br>-movimientosInventario: List<MovimientoInventario><br>-sugerencias: List<SugerenciaProducto><br>-turnos: List<Turno><br>-solicitudes: List<SolicitudCambioTurno><br>-pagos: List<PagoMensualidad> |
| +autenticar(login: String, password: String): Usuario<br>+buscarUsuario(login: String): Usuario<br>+crearEquipo(administrador: Administrador, nombre: String, cupoMaximo: int, deporte: DeporteConjunto, categoria: Categoria): Equipo<br>+crearPerfilJugador(administrador: Administrador, usuario: Usuario): PerfilJugador<br>+darDeBajaEquipo(administrador: Administrador, equipo: Equipo): void<br>+darDeBajaJugador(administrador: Administrador, jugador: PerfilJugador): void<br>+programarPartidoInterno(id: String, local: Equipo, visitante: Equipo, fecha: LocalDate): PartidoInterno<br>+consultarCalendario(jugador: PerfilJugador): List<Partido><br>+consultarEstadisticas(jugador: PerfilJugador): List<EstadisticaPartido><br>+puedeRecibirBebidaCaliente(usuario: Usuario, momento: LocalDateTime): boolean<br>+registrarVenta(venta: Venta, ubicacion: UbicacionInventario, puntosARedimir: int): void<br>+moverProducto(administrador: Administrador, producto: Producto, origen: UbicacionInventario, destino: UbicacionInventario, cantidad: int, fecha: LocalDate): void<br>+reabastecer(administrador: Administrador, producto: Producto, destino: UbicacionInventario, cantidad: int, fecha: LocalDate): void<br>+consultarInventario(ubicacion: UbicacionInventario): List<Existencia><br>+registrarSugerencia(autor: Empleado, descripcion: String, fecha: LocalDate): SugerenciaProducto<br>+hayCobertura(dia: DayOfWeek, hora: LocalTime): boolean<br>-hayCoberturaConCambio(dia: DayOfWeek, hora: LocalTime, turnoCambiado: Turno, nuevoEmpleado: Empleado): boolean<br>+solicitarCambioTurno(solicitante: Empleado, turno: Turno, reemplazo: Empleado, fecha: LocalDate, motivo: String): SolicitudCambioTurno<br>+resolverCambioTurno(administrador: Administrador, solicitud: SolicitudCambioTurno, aprobar: boolean): void |

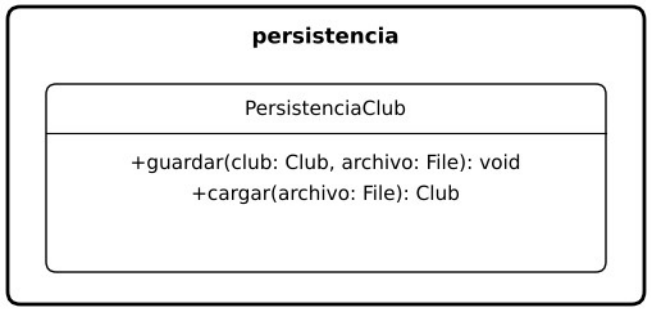
## ADMINISTRACION: TURNOS Y REPORTES



## EXCEPCIONES



**PERSISTENCIA**



# 8. DECISIONES DE DISEÑO

Diseñar es escoger entre varias formas de hacer lo mismo. Cada decisión de esta sección tiene cuatro partes: el **problema** que había que resolver, las **alternativas** que había, la que **escogimos** y lo que **ganamos y sacrificamos** con ella. Ninguna es gratis: siempre se sacrifica algo.

Son las seis decisiones que más cambiaron la estructura del modelo. Para cada una hay una explicación en palabras simples, la ficha y dónde se ve en el código.

### D1. ¿CÓMO REPRESENTAR QUE UN USUARIO JUEGA?

**El problema en palabras simples.** Lo natural era pensar que el jugador es un tipo de socio. Pero en el club también juegan los entrenadores: pueden tener partidos de práctica contra cualquier nivel. Si Jugador fuera una subclase de Socio, un entrenador no podría ser jugador, porque en Java un objeto no puede ser dos subclases a la vez.

| | |
|---|---|
| **Alternativas** | A) Jugador como subclase de Socio.<br>B) Un PerfilJugador aparte, asociado a Usuario, que puede tener cualquier usuario. |
| **Elegida** | B |
| **Ganamos** | Cualquier usuario puede jugar sin dejar de ser lo que es. El perfil agrupa todo lo del jugador: sus inscripciones a equipos y a disciplinas. |
| **Sacrificamos** | Una clase más en el camino: para ir del jugador a su nombre hay que pasar por `getUsuario()`. |
| **Relacionada con** | RN-23 y RN-38. RF-S-03 y RF-S-07. |

**Dónde se ve en el código.** `Usuario` tiene el atributo `perfilJugador` y el método `crearPerfilJugador()`. `Administrador` lo redefine con `@Override` para cumplir RN-38.

### D2. ¿DÓNDE SE REVISAN LAS REGLAS?

**El problema en palabras simples.** Había dos caminos. Uno era que Club revisara todas las reglas, y las demás clases solo guardaran datos (control centralizado). El otro era que cada clase revisara las reglas de sus propios datos, y Club solo las que cruzan partes del modelo (control delegado). Escogimos el delegado. Es el mismo ejemplo que usa la guía con la cafetería: Cafeteria coordina pero no calcula, cada producto sabe su precio.

| | |
|---|---|
| **Alternativas** | A) Control centralizado: Club revisa todo.<br>B) Control delegado: cada clase revisa lo suyo y Club coordina lo que cruza partes. |
| **Elegida** | B |
| **Ganamos** | Cada regla está junto a sus datos: el cupo en Equipo, la edad en Categoria, la modalidad en Instalacion. Cambiar una regla toca una sola clase, y Club no se vuelve un "gestor" que hace todo. |
| **Sacrificamos** | Para seguir una operación completa hay que pasar por varias clases. Club igual quedó grande, porque tiene las listas de todo y las reglas que cruzan partes. |
| **Relacionada con** | RNF de facilidad de cambio. Todas las reglas de dominio. |

**Dónde se ve en el código.** `Equipo.inscribirJugador` revisa cupo, edad y camiseta sin pasar por Club. `Club.programarPartidoInterno` sí está en Club, porque necesita mirar dos equipos a la vez.

### D3. ¿CÓMO TRATAR LO QUE CAMBIA SEGÚN EL TIPO?

**El problema en palabras simples.** Hay cosas que funcionan distinto según el tipo: la tienda cobra IVA y la cafetería impuesto al consumo; una cancha de pádel solo acepta dobles y una de tenis acepta los dos. Se podía guardar un atributo "tipo" y llenar el código de if. Preferimos que cada subclase sepa hacer lo suyo, y que la clase de arriba le pregunte sin saber de qué tipo es (polimorfismo).

| | |
|---|---|
| **Alternativas** | A) Un atributo "tipo" e if o instanceof en cada operación.<br>B) Métodos abstractos que cada subclase redefine. |
| **Elegida** | B |
| **Ganamos** | Un tipo nuevo es una subclase nueva, sin tocar lo que ya funciona. Hay 38 métodos redefinidos con @Override en el código y ningún instanceof en el modelo. |
| **Sacrificamos** | Más clases y métodos pequeños. Para saber qué pasa en un caso hay que mirar la subclase. |
| **Relacionada con** | RN-10, RN-12, RN-17, RN-24, RN-28, RN-29 y RN-32. |

**Dónde se ve en el código.** `Venta.cerrar` llama a `calcularImpuestos(base)` sin saber si es VentaTienda (19 %) o VentaCafeteria (8 %). `Partido.cerrar` llama a `despuesDeCerrar()`, que cada tipo de partido escribe a su manera.

### D4. ¿CÓMO AVISA EL MODELO QUE ALGO NO SE PUEDE HACER?

**El problema en palabras simples.** Cuando alguien intenta algo que rompe una regla, el modelo tiene que decirlo. Se podía devolver false o null, pero entonces no se sabe qué regla falló, y quien llama puede olvidar revisarlo. Preferimos lanzar una excepción que lleve el número de la regla.

| | |
|---|---|
| **Alternativas** | A) Devolver false o null.<br>B) Lanzar una excepción checked con el número de la regla. |
| **Elegida** | B |
| **Ganamos** | El mensaje dice qué regla se rompió, por ejemplo `[RN-04] Edad 22 fuera del rango de Sub-15`. Como son checked, Java obliga a quien llama a manejarlas. El diseño busca validar las condiciones antes de modificar el estado, pero algunas operaciones pueden dejar cambios parciales si ocurre un error durante su ejecución. Este riesgo se identifica en el registro de ventas y queda como una mejora pendiente. |
| **Sacrificamos** | Más código en quien llama (try y catch) y siete clases de excepción. |
| **Relacionada con** | RNF de mensajes claros. |

**Dónde se ve en el código.** `ReglaDominioException` extiende `Exception` y tiene seis subclases. Las preguntas como `estaHabilitado()` sí devuelven boolean, porque preguntar no es fallar.

## D5. ¿CÓMO SE GUARDA EL ESTADO?

**El problema en palabras simples.** El sistema necesita guardar la información del club para poder recuperarla después. Para ello, usamos archivos de texto: cada registro se escribe en una línea y sus datos se separan mediante un delimitador. Al cargar la información, PersistenciaClub lee las líneas, interpreta sus datos y reconstruye los objetos y sus relaciones. Este mecanismo requiere definir cómo se representa cada tipo de objeto en el archivo y cómo se recuperan sus relaciones.

| | |
|---|---|
| **Alternativas** | A) Serializar el objeto Club completo en un archivo binario mediante la serialización de Java.<br>B) Guardar los datos en archivos de texto y reconstruir los objetos y sus relaciones al cargarlos. |
| **Elegida** | B |
| **Ganamos** | El archivo puede leerse como texto y la persistencia queda centralizada en PersistenciaClub, sin mezclar la lectura y escritura de archivos con las reglas del modelo. |
| **Sacrificamos** | Hay que convertir los datos de cada objeto a un formato de texto y reconstruir los objetos y sus relaciones al cargar. Si cambia el formato de los registros, también puede ser necesario actualizar el código de lectura y escritura. |
| **Relacionada con** | Restricción de archivos fuera de src. RNF de facilidad de cambio. |

**Dónde se ve en el código.** PersistenciaClub centraliza las operaciones de guardar y cargar. El método guardar escribe los registros en un archivo de texto y el método cargar los lee e interpreta para reconstruir el estado del club.

### D6. ¿QUÉ ES LA RAÍZ DEL SISTEMA?

**El problema en palabras simples.** Todo lo que existe en el club tiene que estar en algún lado. Escogimos una sola clase, Club, que tiene una lista de cada cosa. No es una superclase, porque nadie hereda de ella. Es el objeto del que cuelga todo lo demás.

| | |
|---|---|
| **Alternativas** | A) Listas sueltas en cada menú.<br>B) Una clase Club de la que cuelga todo. |
| **Elegida** | B |
| **Ganamos** | Un solo objeto para guardar y cargar, un solo lugar donde buscar y un punto de entrada claro para las operaciones que cruzan partes. |
| **Sacrificamos** | Club es la clase más grande del modelo, con 17 listas. Para que no se vuelva un gestor, las reglas de cada parte se quedan en su clase (D2) y los reportes van en GeneradorReportes. |
| **Relacionada con** | RF de consulta e historial. Persistencia. |

**Dónde se ve en el código.** Los 17 atributos `List<...>` de `Club`. Club guarda solo los partidos internos: los de competencia viven en su competencia y los de práctica en su reserva.

# 9. RIESGOS Y CONSISTENCIA

### PERSISTENCIA Y MANEJO DE ERRORES

- El club se guarda en datos/club.txt, fuera de src. Si la carpeta no existe, se crea.
- Si club.txt no existe, la aplicación carga datos/club_ejemplo.txt. Si el archivo está dañado, cargar lanza una IOException que dice en qué línea falló. La aplicación muestra ese mensaje y se cierra sin tocar el archivo. Para seguir, se arregla esa línea o se borra datos/club.txt y la aplicación arranca con el club de ejemplo.
- Toda operación que cambia algo revisa sus reglas antes de modificar cualquier cosa. Así un rechazo no deja relaciones a medias, como un jugador en la lista del equipo pero no en la suya.
- Los valores fijos del enunciado son constantes con nombre: `TASA_IVA`, `TASA_CONSUMO`, `PORCENTAJE_PUNTOS`, `RANKING_INICIAL`, `PUNTOS_EN_JUEGO`, `PARTIDOS_POR_SANCION`.

### RIESGOS CONOCIDOS

Lo que sabemos que es débil en el diseño actual, qué puede pasar y qué haríamos si hay que arreglarlo.

| Riesgo | Qué puede pasar | Qué haríamos |
|---|---|---|
| **Los getters del Club devuelven la lista real** | Si devolviera la lista real, alguien de afuera podría hacer club.getEquipos().add(...) y saltarse las reglas de crearEquipo. | Ya está hecho: cada getter del Club devuelve una lista que no se puede modificar (Collections.unmodifiableList). |
| **Varios métodos reciben un `Administrador` que no usan** | Parece un parámetro de sobra. Lo usamos para que solo un administrador pueda llamarlos, pero no se revisa nada más. | Dejarlo así y explicarlo, o revisar que el administrador esté activo. |
| **El rol se compara como texto** | `hayCobertura` cuenta entrenadores con `getRol().equals("Entrenador")`. Si el texto cambia en una subclase, deja de contar. | Cambiar el rol por un enum. |
| **La cobertura se revisa cada 30 minutos** | Si un turno empezara a las 8:15, el hueco entre las 8:00 y las 8:15 no se revisa. | Revisar en cada hora donde empieza o termina un turno. |
| **Un producto repetido en dos líneas de una venta** | Cada línea se valida por separado contra las existencias: la validación podría pasar y fallar después, al retirar. | Sumar las cantidades por producto antes de validar. |
| **Club es grande** | Tiene 17 listas y las operaciones que cruzan partes. Podría empezar a absorber reglas que son de otra clase. | Mantener D2: cada regla nueva va en la clase que tiene sus datos. |
| **El archivo es de texto** | Se puede editar a mano, y un error de edición rompe la carga. Si una clase cambia, cambia también el formato de su línea. | Borrar datos/club.txt: la aplicación vuelve a arrancar con el club de ejemplo. El archivo no se sube al repositorio (está en .gitignore). |

## CAMBIOS PREVISTOS

Si pasa algo de esto, qué parte del código cambia. La idea es que cada cambio toque pocas clases.

| Si pasa esto | Cambia |
|---|---|
| **Cambiar los menús de consola** | Solo cambia el paquete interfaz. El modelo ya tiene un método por cada requerimiento. |
| **Interfaz gráfica (Proyecto 3)** | Igual: el modelo no imprime nada. |
| **Un deporte de conjunto nuevo** | Un objeto DeporteConjunto nuevo y una subclase de EstadisticaPartido. |
| **Un tipo de instalación nuevo** | Una subclase de Instalacion con su `admite` y su `prohibeBebidaCaliente`. |
| **Un punto de venta con otro impuesto** | Una subclase de Venta con su `calcularImpuestos` y su `getRubro`. |
| **Guardar en una base de datos u otro formato** | Solo PersistenciaClub. |
| **Las amarillas se cuentan por competencia** | InscripcionEquipo y la nómina de la competencia (NominaCompetencia o RenglonNomina). |
| **Se agrega la habilitación médica** | Una clase nueva HabilitacionMedica, un método del fisioterapeuta, y `InscripcionEquipo.estaHabilitado` tendría que revisarla. |
| **Se revisa el deporte de la estadística** | `Partido.registrarEstadistica` y un método nuevo en cada subclase de EstadisticaPartido. |

## LAS CUATRO RECOMENDACIONES DE LA GUÍA

La guía del documento de diseño pide revisar el diseño contra cuatro recomendaciones. Así las cumplimos:

| Recomendación | Cómo la cumplimos |
|---|---|
| **Reducir la complejidad** | Cada clase se explica en una frase (sección 3) y cada método hace una cosa. Lo más largo, el cálculo de una venta y el ranking, está en un solo método cada uno, con los pasos comentados. |
| **Pensar en los cambios** | La tabla de cambios previstos: lo que cambia por tipo está en subclases, y la interfaz y el archivo están separados del modelo. |
| **Reducir el acoplamiento** | Partido no conoce sus subclases, Venta no sabe qué impuesto cobra y el modelo no conoce la interfaz ni el archivo. |
| **Inyección de dependencias** | GeneradorReportes recibe el Club en su constructor y PersistenciaClub recibe el archivo como parámetro en guardar y cargar. Club no recibe dependencias porque es la raíz. |

## RECOMENDACIONES DE LA PROFESORA

Espacio para lo que nos recomiende la profesora en la revisión, y para seguir cada cambio hasta el código.

| Recomendación | Qué parte del diseño toca | Qué cambiamos | Hecho |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

**Cómo llevar una recomendación al código:**

1. Ubicarla: ¿es sobre una clase, una relación, una regla o una de las decisiones de la sección 8?
2. Si cambia una decisión, actualizar su ficha: qué alternativa queda ahora y qué ganamos y sacrificamos.
3. Cambiar el código en la clase que tiene esos datos (la tabla de responsabilidades dice cuál es).
4. Correr la aplicación, recorrer los tres menús con datos/club_ejemplo.txt y revisar que todo siga saliendo como se espera.
5. Volver a generar los diagramas de las secciones 6 y 7, para que sigan iguales al código.

## COHERENCIA ENTRE DIAGRAMAS Y CÓDIGO

- Los diagramas de clases (secciones 6 y 7) se generaron leyendo el código, así que los nombres de clases, atributos, métodos y firmas son los mismos.
- Los paquetes del código son los de la sección 2, con los nombres de la guía de convenciones del curso: `uniandes.dpoo.proyecto1.<parte>`.
- La tabla de requerimientos de la sección 1 apunta a métodos que existen en el código.
- Si el código cambia, se vuelven a generar los diagramas.

# ANEXO: USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

## 1. MARCO Y NIVEL DE USO

Esta entrega se desarrolló bajo el Nivel 3 de los lineamientos de la Universidad: la IA generativa se usa para mejorar la claridad y calidad de ideas del estudiante, y su uso se documenta en un anexo.

Las decisiones de diseño (secciones 3, 4 y 8), la estructura de clases y las funcionalidades críticas son del grupo. La IA se usó para refinar, revisar y apoyar trabajo que el grupo ya había producido.

## 2. HERRAMIENTAS UTILIZADAS

| Herramienta | Uso general |
|---|---|
| **Claude (Anthropic), en chat** | Revisión del documento de diseño contra el enunciado, el análisis y el código, y apoyo en la claridad de la redacción. |
| **Asistente de chat de IA generativa** | Refinamiento de código ya escrito, depuración, plantillas y datos de prueba. |
| **Generación de diagramas a partir del código** | Elaboración de los diagramas de clases de las secciones 6 y 7, verificados contra el código. |
| **Eclipse (IDE)** | Generación de getters, setters y constructores. |

## 3. CRITERIOS APLICADOS

1. La IA no escribió funcionalidades críticas: reglas de dominio (RN-01 a RN-38), cálculo de ventas e impuestos, ranking, sanciones, cobertura de turnos y persistencia.

2. La IA se usó para refinar código ya creado por el grupo, depurar errores, crear plantillas y datos de prueba, y mejorar la redacción.

3. Getters, setters y constructores se consideran código repetitivo y no una funcionalidad crítica.

4. Toda salida fue leída, contrastada y ajustada por el grupo antes de incorporarse.

5. Cada uso queda registrado con su propósito y su resultado.

## 4. REGISTRO DE USOS

### 4.1 DOCUMENTACIÓN

| # | Herramienta | Propósito | Solicitud | Qué se hizo con la salida | Sección |
|---|---|---|---|---|---|
| **D1** | Claude | Detectar faltantes e inconsistencias del documento de diseño | "Qué le hace falta al documento de diseño??? Míralo y dime todo lo que hace falta" (con análisis, diseño y enunciado adjuntos) | El grupo contrastó la lista con el enunciado y decidió qué corregir. Descartó la observación sobre diagramas ausentes en las secciones 6 y 7, porque la IA solo leyó el texto y no las imágenes. | Todo el documento |
| **D2** | Asistente de chat | Mejorar claridad de redacción de secciones ya escritas por el grupo | Solicitudes de revisar claridad y consistencia de títulos, tablas y numeración | Se aplicaron ajustes de forma. El contenido técnico no se modificó. | Secciones 1 a 4 y 8 |
| **D3** | Asistente de chat | Contrastar tablas de requerimientos y reglas con el enunciado | Solicitudes de revisar coherencia entre RF, RN y métodos | El grupo verificó cada correspondencia en el código y corrigió lo necesario. | Secciones 1 y 4 |
| **D4** | Claude | Revisar que el documento coincida con el código antes de entregar | "Revisa todo de nuevo y ayúdame a cerrar la entrega" (con el documento, el código y la rúbrica) | Se corrigieron el ejemplo de punta a punta, la decisión D5, lo que pasa con un archivo dañado, la tabla de clases (OpcionesComunes) y se agregó el enlace del video. El grupo revisó cada cambio contra el código. | Secciones 1 a 4, 8 y 9 |

### 4.2 DIAGRAMAS

| # | Herramienta | Propósito | Método | Qué se hizo con la salida | Verificación |
|---|---|---|---|---|---|
| **G1** | Generación a partir del código | Diagramas de clases de alto nivel (sección 6) | Se generaron leyendo el código fuente | El grupo revisó que se leyeran bien y los reorganizó en seis partes. | Se compararon clases y relaciones contra el código en Eclipse. |
| **G2** | Generación a partir del código | Diagramas de clases detallados (sección 7) | Se generaron leyendo el código fuente | El grupo comprobó atributos, métodos y firmas. | Se compararon nombres y firmas contra el código en Eclipse. |
| **G3** | Asistente de chat | Apoyo en la sintaxis de los diagramas de secuencia (sección 5) | Solicitudes de sintaxis y formato | El grupo definió los escenarios y los mensajes, y revisó cada uno contra los métodos reales. | Cada mensaje se contrastó con el método correspondiente. |
| **G4** | Claude | Diagrama de arquitectura (sección 2) y recuadro de PersistenciaClub (sección 7) | Se generaron a partir de los paquetes y métodos del código | El grupo los revisó y los dejó en blanco y negro, como el resto del documento. | Se comprobó que cada nombre existe en el código. |

### 4.3 CÓDIGO

| # | Herramienta | Propósito | Qué se hizo con la salida | Clases afectadas |
|---|---|---|---|---|
| **C1** | Eclipse y asistente de chat | Getters, setters y constructores | Se incorporaron tras revisión. | Clases del modelo |
| **C2** | Asistente de chat | Depuración de errores de compilación y de ejecución | El grupo entendió la causa y aplicó la corrección. | Clases del modelo y menús (paquete interfaz) |
| **C3** | Asistente de chat | Datos de prueba y plantillas | Se ajustaron a las reglas y escenarios del club. | datos/club_ejemplo.txt y Entrada |
| **C4** | Asistente de chat | Refinar código ya escrito: nombres, comentarios y duplicación | Se aplicaron los cambios que el grupo consideró correctos. | Clases del modelo y menús |
| **C5** | Asistente de chat | Plantilla de la estructura de los menús de cada tipo de usuario | El grupo escribió el contenido de cada opción y sus casos de rechazo. | MenuSocio, MenuEmpleado, MenuAdministrador |
| **C6** | Asistente de chat | Consultas sobre la estructura: cómo unir las clases, dónde usar herencia y @Override, y qué significa tener una clase raíz desde donde funcionan los menús | El modelo del análisis tenía 58 clases. En el código quedaron 75 clases y 9 enum en 10 paquetes, con 38 métodos redefinidos. La estructura pasó por varias versiones en GitHub, con ramas y pushes: algunas se borraron y otras ya estaban bien divididas por paquetes. | Club y las jerarquías de herencia |
| **C7** | Asistente de chat | Errores de importaciones y de formato. Qué es Serializable y cómo ayudaría a guardar la clase raíz | Se corrigieron los imports. No se usó Serializable: se escogió un archivo de texto plano (decisión D5). | Varias clases y PersistenciaClub |

### 4.4 VIDEO DE DEMOSTRACIÓN

| # | Herramienta | Propósito | Qué se hizo con la salida |
|---|---|---|---|
| **V1** | Claude | Ordenar el recorrido de la grabación por tipo de usuario y hacer la portada | El grupo grabó y editó el video (Game Bar y Clipchamp) y escribió los subtítulos. |

## 5. LO QUE NO SE HIZO CON IA

| Componente | Autoría |
|---|---|
| **Modelo de dominio y análisis (Entrega 1)** | El grupo, con el uso declarado en el anexo de esa entrega (Nivel 2) |
| **Decisiones D1 a D6 y sus alternativas** | El grupo |
| **Reglas de dominio y excepciones** | El grupo |
| **Lógica de ventas, impuestos, puntos y descuentos** | El grupo |

| Componente | Autoría |
|---|---|
| **Ranking, sanciones y cobertura de turnos** | El grupo |
| **Persistencia (PersistenciaClub)** | El grupo |

## 6. VERIFICACIÓN Y APROPIACIÓN

• Código: cada fragmento se leyó, se compiló y se probó con los tres menús antes de aceptarse.

• Documentación: cada afirmación se contrastó con el enunciado, el análisis de la Entrega 1 y el código.

• Diagramas: se comprobó que nombres, atributos y firmas coincidieran con el código.

• Autoría: todos los integrantes pueden explicar cualquier clase, método o decisión en la sustentación.

## 7. ERRORES Y LÍMITES ENCONTRADOS

| Situación | Qué pasó | Cómo se resolvió |
|---|---|---|
| **Revisión del documento de diseño** | La IA señaló que las secciones 6 y 7 no tenían diagramas, porque procesó solo el texto y no las imágenes. | El grupo comprobó el documento y mantuvo los diagramas. |
| **Sugerencias de código** | Algunas sugerencias no respetaban las decisiones de diseño del grupo, como el control delegado de D2. | El grupo las descartó o las adaptó para mantener las reglas en la clase que tiene los datos. |
| **Revisión de cierre** | La IA dijo que el documento no tenía diagramas de secuencia, porque leyó solo el texto, y agregó tres que repetían los de la sección 5. | El grupo los quitó y dejó los diagramas de la sección 5, hechos por el grupo. |

## 8. DECLARACIÓN DEL GRUPO

Declaramos que el uso de IA en esta entrega se limitó a lo descrito en este anexo, que corresponde al Nivel 3, y que no se usó para generar funcionalidades críticas. Entendemos que el profesor puede pedir una sustentación adicional.

Grupo 4: Manuela Leon, Samuel Fonseca, Camilo Castro